Reg. No._____ Name:_____

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

SIXTH SEMESTER B.TECH DEGREE MODEL EXAMINATION, APRIL 2018

Course Code: CS 306

Course Name: Computer Networks

Max. Marks: 100 Duration: 3 Hours

Part A (4x3=12)

(Answer All Questions. Each caries 3 marks)

1. Explain IEEE 802.5 standard?
2. Explain the terms interface ,services and protocols.
3. Explain the states of operation of PPP protocol.
4. Distinguish between Fast Ethernet and Gigabit Ethernet standards.

Part B (2x9=18)

(Answer any two. Each caries 9 marks)

5. Identify the layers in OSI reference model and illustrate their functions.
6. How packet loss is detected in Go-Back-N ARQ technique and Selective Repeat protocol.
7. Explain the architecture and communication model of HDLC protocol.

Part C (4x3=12)

(Answer All Questions. Each caries 3 marks)

8. Explain the different classes of IP addresses.
9. Explain optimality principle.
10. Explain the concept of flooding.
11. Draw the header structure of IPv4 protocol and explain the fields.

.

Part D (2x9=18)

(Answer any two. Each caries 9 marks)

12. Discuss about the techniques to improve QoS in internetworking.
13. Illustrate the routing procedure in mobile networks.
14. Explain the various congestion control techniques.
15. Explain Link State routing algorithm.

Part E (4x10=40)

(Answer any four. Each caries 10 marks)

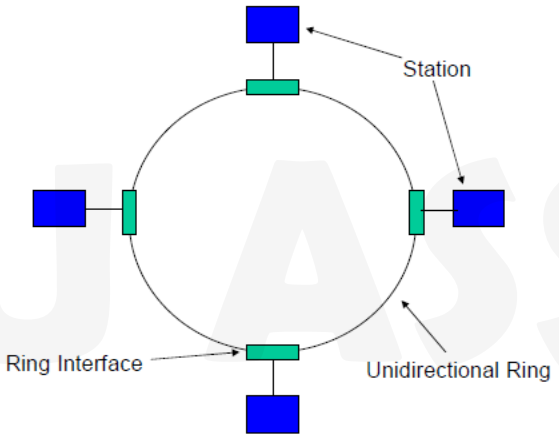
16. Illustrate the working of OSPF protocols.
17. Explain how control messages are sent using ICMP protocol.
18 a. Give the format of TCP header and discuss the relevance of various fields
  b. How transport layer connection is established in TCP? Illustrate with state diagrams.
19. Explain the process of address resolution by using ARP and RARP protocol.
20. Explain how network is managed by using SNMP protocol.
21. Explain the architecture of World Wide Web.

Part A **(4x3=12)**

(Answer All Questions. Each caries 3 marks)

1. Explain IEEE 802.5 standard?

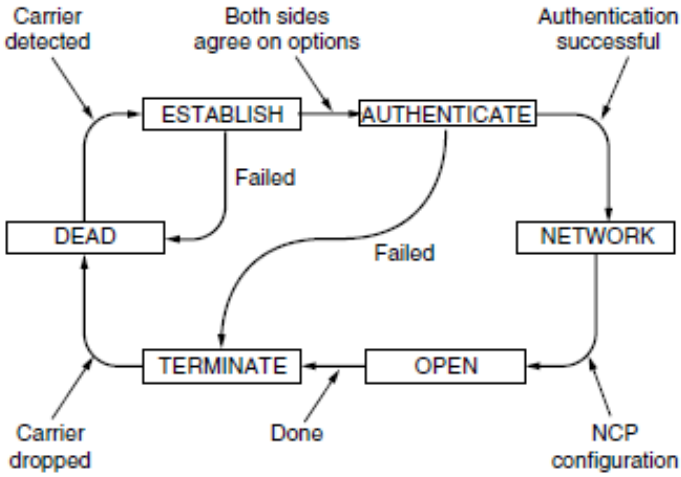
- Ring is not a broadcast medium but a collection of point-to-point links forming a circle.
- Token is a special Frame which gives the holder station the "Right to Transmit"
- All stations are connected are organized in a Logical ring
- Frames are passed from the Predecessor to the successor after a specified time interval
- When there is no data to be sent the token circulates around the logical ring
- Whenever a station has data to send, it waits for a token to arrive
- Station then captures the token and keeps transmitting data until allocated time for keeping the token expires
- After the specified time the token must be passed on to the successor
- Fair operation with an upper bound on channel access
- Channel access problem is solved with the help of a special frame called a "Token"



2. Explain the terms interface ,services and protocols .

To reduce the complexity of design , most networks are organized as a stack of **Layers.** When layer n on one machine carries on a conversation with layer n on another machine, the rules and conventions used in this conversation are collectively known as the layer n protocol. Basically, a **protocol** is an agreement between the communicating parties on how communication is to proceed. The interface defines which primitive operations and services the lower layer makes available to the upper one. A service is a set of primitives (operations) that a layer provides to the layer above it. The service defines what operations the layer is prepared to perform on behalf of its users, but it says nothing at all about how these operations are implemented. A service relates to an interface between two layers, with the lower layer being the service provider and the upper layer being the service user. A protocol, in contrast, is a set of rules governing the format and meaning of the packets, or messages that are exchanged by the peer entities within a layer. Entities use protocols to implement their service definitions.

3. Explain the states of operation of PPP protocol.

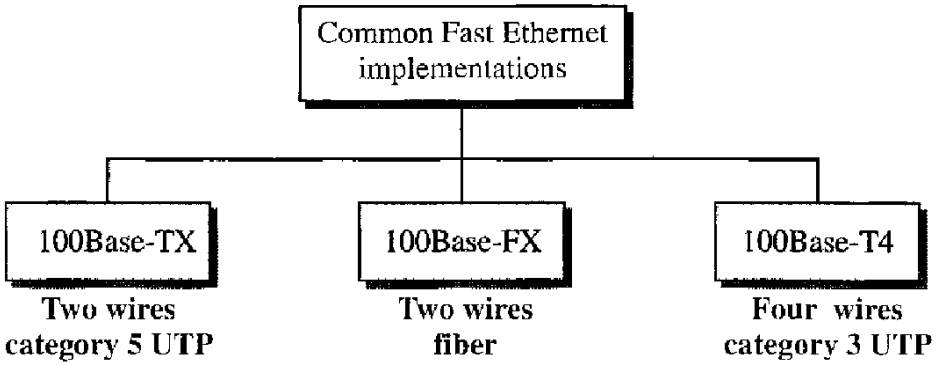


The link starts in the DEAD state, which means that there is no connection at the physical layer. When a physical layer connection is established, the link moves to ESTABLISH. At this point, the PPP peers exchange a series of LCP packets, each carried in the Payload field of a PPP frame, to select the PPP options for the link from the possibilities mentioned above. The initiating peer proposes options, and the responding peer either accepts or rejects them, in whole or part. The responder can also make alternative proposals. If LCP option negotiation is successful, the link reaches the AUTHENTICATE state. Now the two parties can check each other's identities, if desired. If authentication is successful, the NETWORK state is entered and a series of NCP packets are sent to configure the network layer. Once OPEN is reached, data transport can take place. It is in this state that IP packets are carried in PPP frames across the SONET line. When data transport is finished, the link moves into the TERMINATE state, and from there it moves back to the DEAD state when the physical layer connection is dropped.

4. Distinguish between Fast Ethernet and Gigabit Ethernet standards.

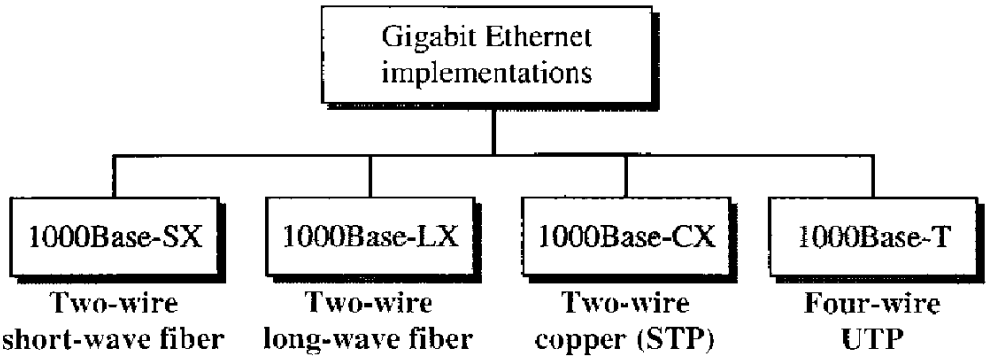
The goal of fast Ethernet can be summarized as

1. Upgrade the data rate to 100 Mbps.
2. Make it compatible with Standard Ethernet.
3. Keep the same 48-bit address.
4. Keep the same frame format.
5. Keep the same minimum and maximum frame lengths.



Gigabit Ethernet design can be summarized as follows:

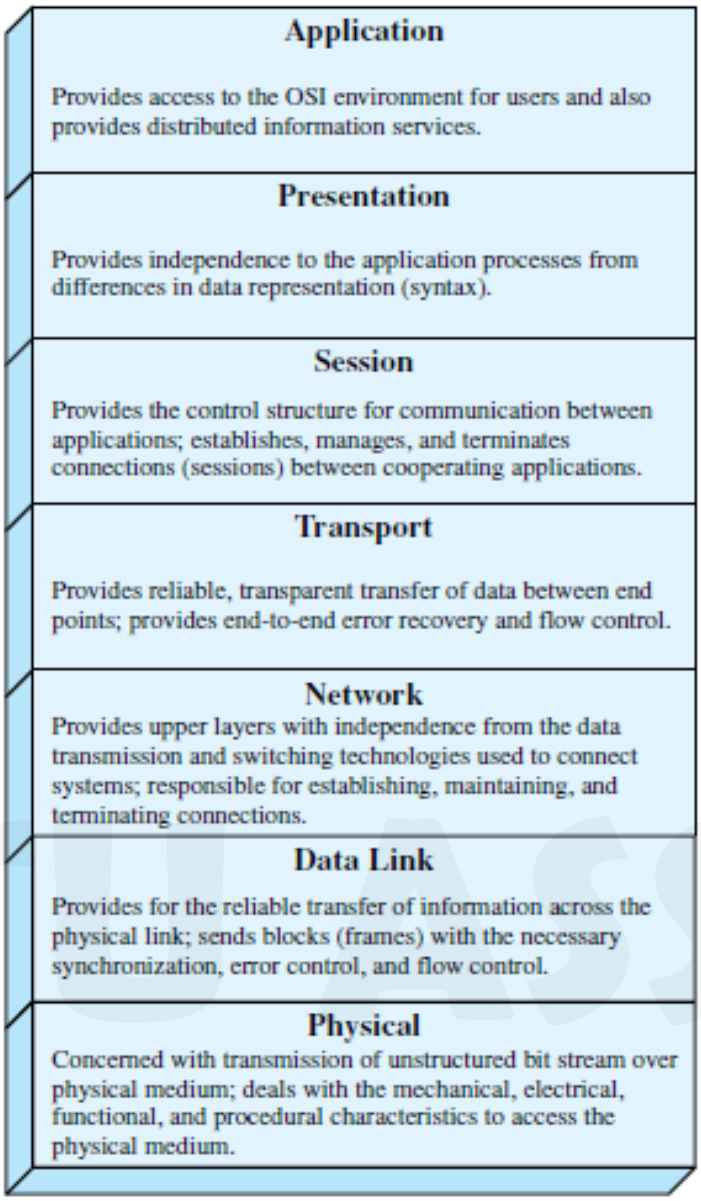
1. Upgrade the data rate to 1 Gbps.
2. Make it compatible with Standard or Fast Ethernet.
3. Use the same 48-bit address.
4. Use the same frame format.
5. Keep the same minimum and maximum frame lengths.
6. To support autonegotiation as defined in Fast Ethernet.



Part B **(2x9=18)**

(Answer any two. Each caries 9 marks)

5. Identify the layers in OSI reference model and illustrate their functions.



6. How packet loss is detected in Go-Back-N ARQ technique and Selective Repeat protocol.

- detection and correction of errors such as:
  - lost frames
  - damaged frames
- common techniques use:
  - error detection
  - positive acknowledgment
  - retransmission after timeout
  - negative acknowledgement & retransmission
- **Automatic Repeat Request (ARQ)** collective name for such error control mechanisms, including :stop and wait, go back N and selective reject (selective retransmission)

  1. Go-Back-N is based on sliding window

     - if no error, ACK as usual
     - use window to control number of outstanding frames

- if error, reply with rejection
- discard that frame and all future frames until error frame received correctly
- transmitter must go back and retransmit that frame and all subsequent frames

Damaged Frame

- error in frame i so receiver rejects frame i
- transmitter retransmits frames from i

Lost Frame

- frame i lost and either
- transmitter sends i+1 and receiver gets frame i+1 out of seq and rejects frame i
- or transmitter times out and send ACK with P bit set which receiver responds to with ACK i
- transmitter then retransmits frames from i

Damaged Acknowledgement

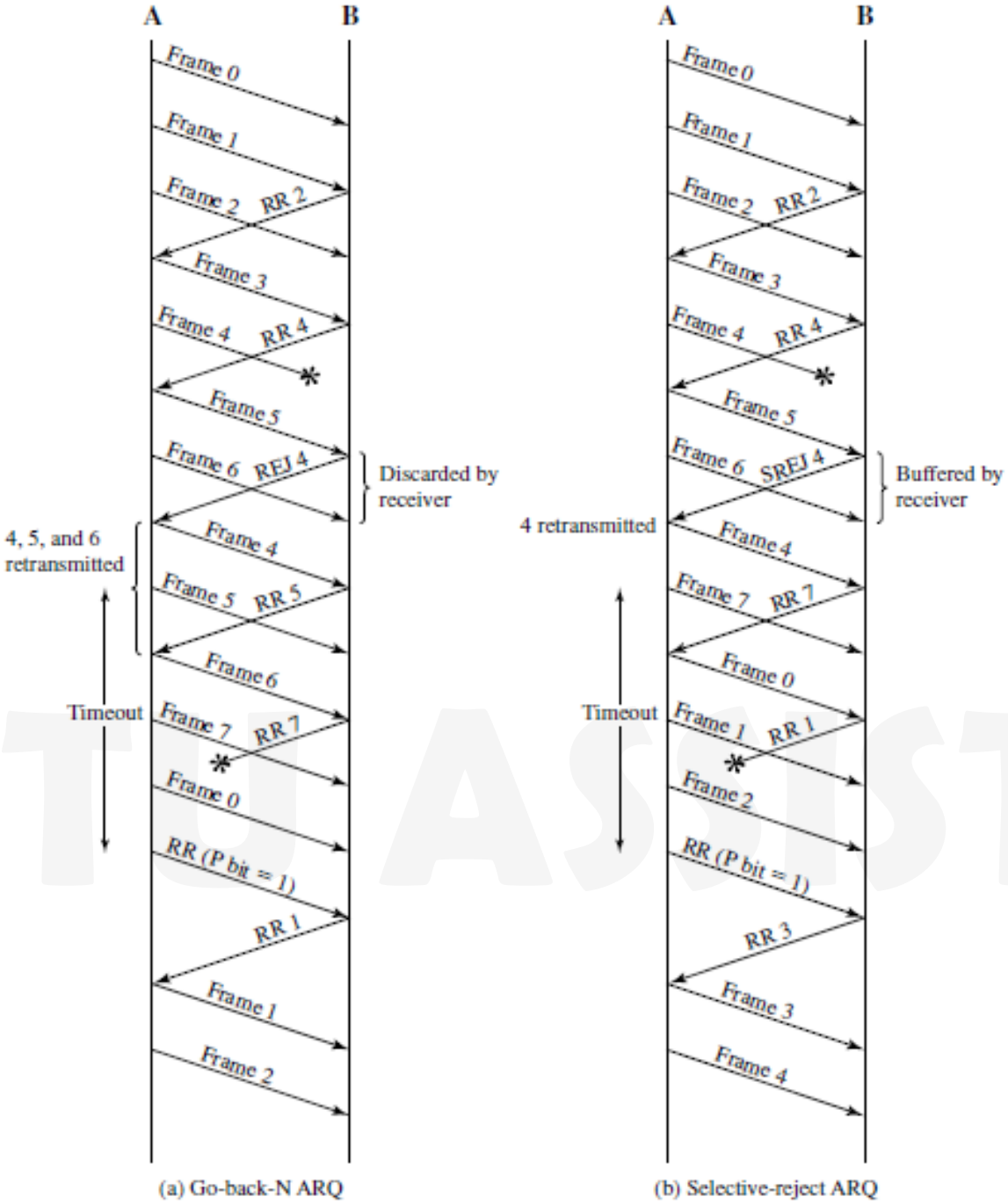
- receiver gets frame i, sends ack (i+1) which is lost
- acks are cumulative, so next ack (i+n) may arrive before transmitter times out on frame i
- if transmitter times out, it sends ack with P bit set
- can be repeated a number of times before a reset procedure is initiated

Damaged Rejection

- reject for damaged frame is lost
- handled as for lost frame when transmitter times out

2.Selective Reject is also called selective retransmission

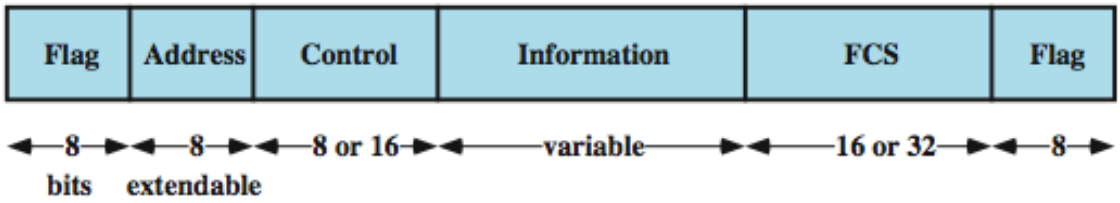
- only rejected frames are retransmitted
- subsequent frames are accepted by the receiver and buffered
- minimizes retransmission
- receiver must maintain large enough buffer
- more complex logic in transmitter
- hence less widely used
- useful for satellite links with long propagation delays



7. Explain the architecture and communication model of HDLC protocol.

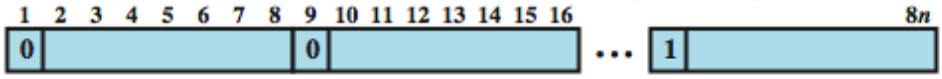
- High Level Data Link Control (HDLC) is an important data link control protocol
- specified as ISO 33009, ISO 4335
- station types:
  - Primary - controls operation of link
  - Secondary - under control of primary station
  - Combined - issues commands and responses
- link configurations
  - Unbalanced - 1 primary, multiple secondary
  - Balanced - 2 combined stations

- Normal Response Mode (NRM)
  - unbalanced config, primary initiates transfer
  - used on multi-drop lines, eg host + terminals
- Asynchronous Balanced Mode (ABM)
  - balanced config, either station initiates transmission, has no polling overhead, widely used
- Asynchronous Response Mode (ARM)
  - unbalanced config, secondary may initiate transmit without permission from primary, rarely used
- synchronous transmission of frames
- single frame format used



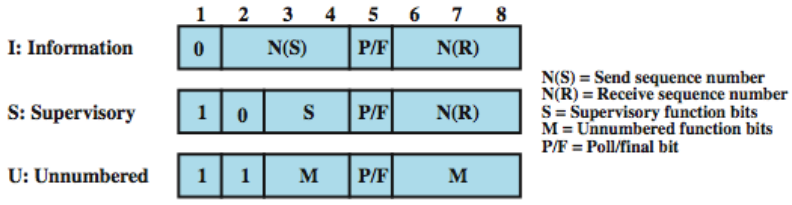
(a) Frame format

- Address field identifies secondary station that sent or will receive frame
  - usually 8 bits long
  - may be extended to multiples of 7 bits
  - LSB indicates if is the last octet (1) or not (0)
  - all ones address 11111111 is broadcast



(b) Extended Address Field

- Control Field is different for different frame type
  - Information - data transmitted to user (next layer up)
    - Flow and error control piggybacked on information frames
  - Supervisory - ARQ when piggyback not used
  - Unnumbered - supplementary link control
- first 1-2 bits of control field identify frame type



(c) 8-bit control field format

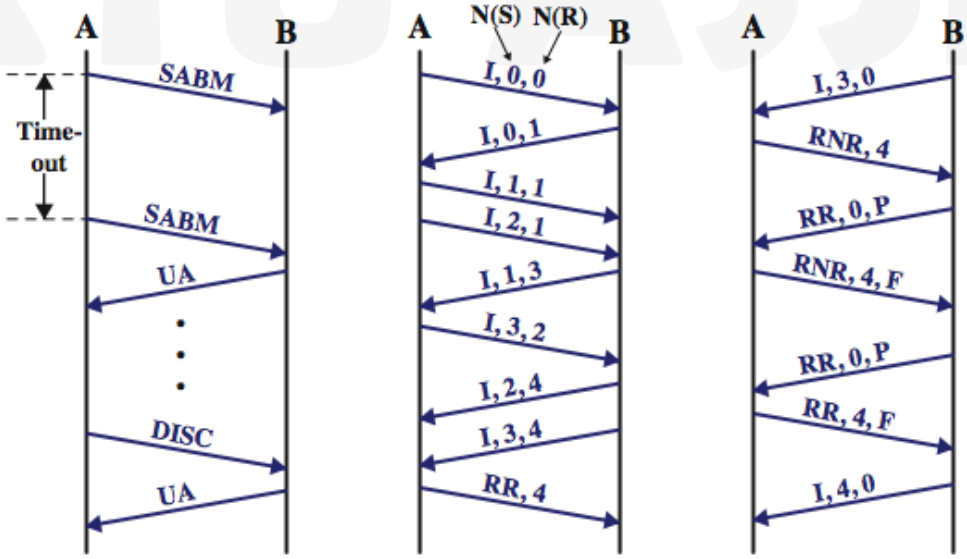
- use of Poll/Final bit depends on context
- in command frame is P bit set to1 to solicit (poll) response from peer
- in response frame is F bit set to 1 to indicate response to soliciting command
- seq number usually 3 bits
  - can extend to 8 bits as shown below

(d) 16-bit control field format

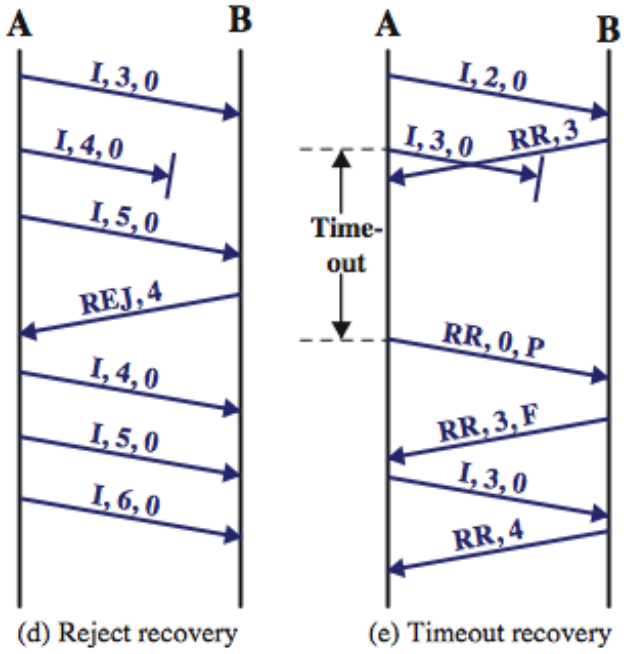
- Information Field
  - in information and some unnumbered frames
  - must contain integral number of octets
  - variable length
- Frame Check Sequence Field (FCS)
  - used for error detection
  - either 16 bit CRC or 32 bit CRC
- HDLC operation consist of exchange of information, supervisory and unnumbered frames

- have three phases
  - initialization
    - by either side, set mode & seq
  - data transfer
    - with flow and error control
    - using both I & S-frames (RR, RNR, REJ, SREJ)
  - disconnect
    - when requested or fault noted

Name	Command/ Response	Description
Information (I)	C/R	Exchange user data
Supervisory (S)		
Receive ready (RR)	C/R	Positive acknowledgment; ready to receive I-frame
Receive not ready (RNR)	C/R	Positive acknowledgment; not ready to receive
Reject (REJ)	C/R	Negative acknowledgment; go back N
Selective reject (SREJ)	C/R	Negative acknowledgment; selective reject
Unnumbered (U)		
Set normal response/extended mode (SNRM/SNRME)	C	Set mode; extended = 7-bit sequence numbers
Set asynchronous response/extended mode (SARM/SARME)	C	Set mode; extended = 7-bit sequence numbers
Set asynchronous balanced/extended mode (SABM, SABME)	C	Set mode; extended = 7-bit sequence numbers
Set initialization mode (SIM)	C	Initialize link control functions in addressed station
Disconnect (DISC)	C	Terminate logical link connection
Unnumbered Acknowledgment (UA)	R	Acknowledge acceptance of one of the set-mode commands
Disconnected mode (DM)	R	Responder is in disconnected mode
Request disconnect (RD)	R	Request for DISC command
Request initialization mode (RIM)	R	Initialization needed; request for SIM command
Unnumbered information (UI)	C/R	Used to exchange control information
Unnumbered poll (UP)	C	Used to solicit control information
Reset (RSET)	C	Used for recovery; resets N(R), N(S)
Exchange identification (XID)	C/R	Used to request/report status
Test (TEST)	C/R	Exchange identical information fields for testing
Frame reject (FRMR)	R	Report receipt of unacceptable frame



(a) Link setup and disconnect (b) Two-way data exchange (c) Busy condition



(d) Reject recovery (e) Timeout recovery

Part C **(4x3=12)**

(Answer All Questions. Each caries 3 marks)

8. Explain the different classes of IP addresses.

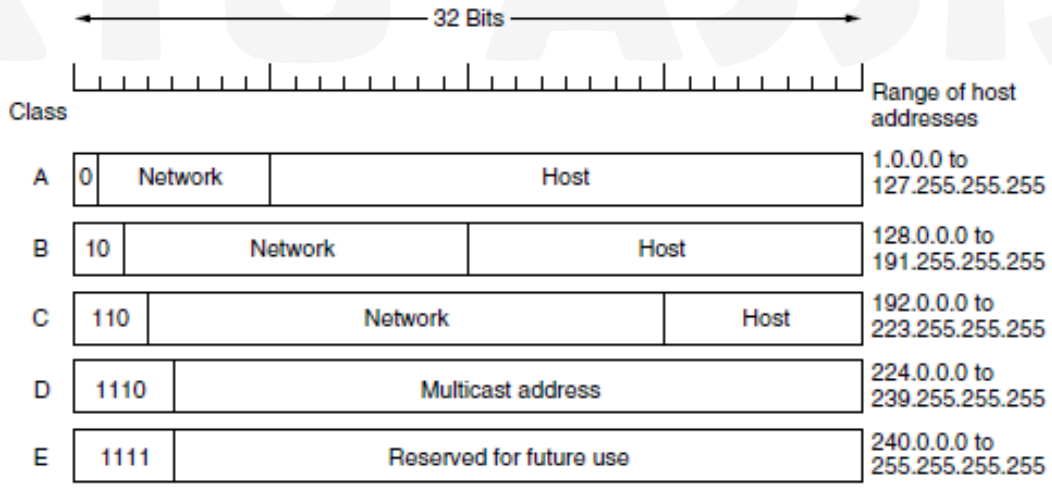
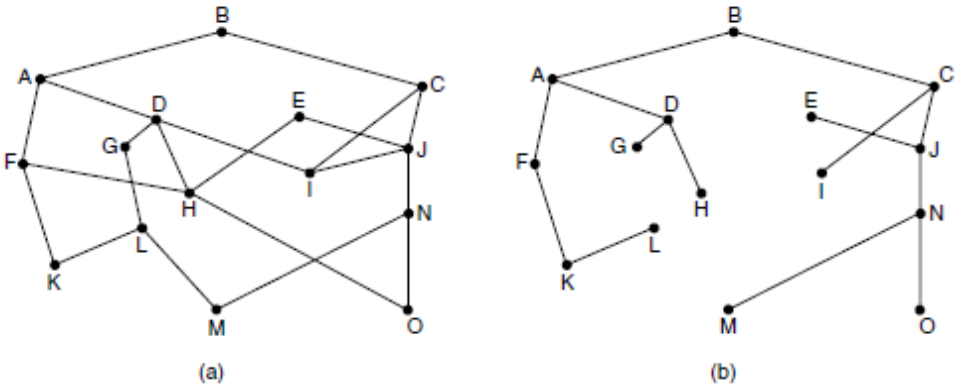


Figure 5-53. IP address formats.

9. Explain optimality principle.

Optimality principle states that if router J is on the optimal path from router I to router K, then the optimal path from J to K also falls along the same route. To see this, call the part of the route from I to J r1 and the rest of the route r 2. If a route better than r 2 existed from J to K, it could be concatenated with r 1 to improve the route from I to K, contradicting our statement that r 1r 2 is optimal. As a direct consequence of the optimality principle, we can see

that the set of optimal routes from all sources to a given destination form a tree rooted at the destination. Such a tree is called a **sink tree** and is illustrated in Fig.,where the distance metric is the number of hops. The goal of all routing algorithms is to discover and use the sink trees for all routers.



10. Explain the concept of flooding.

Flooding is the technique in which every incoming packet is sent out on every outgoing line except the one it arrived on. Flooding obviously generates vast numbers of duplicate packets, in fact, an infinite number unless some measures are taken to damp the process. One such measure is to have a hop counter contained in the header of each packet that is decremented at each hop, with the packet being discarded when the counter reaches zero. Ideally, the hop counter should be initialized to the length of the path from source to destination. If the sender does not know how long the path is, it can initialize the counter to the worst case, namely, the full diameter of the network. Flooding with a hop count can produce an exponential number of duplicate packets as the hop count grows and routers duplicate packets they have seen before. A better technique for damming the flood is to have routers keep track of which packets have been flooded, to avoid sending them out a second time.

11. Draw the header structure of IPv4 protocol and explain the fields.

The Version field keeps track of which version of the protocol the datagram belongs to. IHL, is provided to tell how long the header is, in 32-bit words. The minimum value is 5, which applies when no options are present. The maximum value of this 4-bit field is 15, which limits the header to 60 bytes, and thus the Options field to 40 bytes. The Differentiated services field was intended to distinguish between different classes of service. The Total length includes everything in the datagram—both header and data. The maximum length is 65,535 bytes. The Identification field is needed to allow the destination host to determine which packet a newly arrived fragment belongs to. DF stands for Don't Fragment. It is an order to the routers not to fragment the packet. MF stands for More Fragments. All fragments except the last one have this bit set. The Fragment offset tells where in the current packet this fragment belongs. The TTL (Time to live) field is a counter used to limit packet lifetimes. The Protocol field tells it which transport process to give the packet to. Since the header carries vital information such as addresses, it rates its own checksum for protection, the Header checksum. The Security option tells how secret the information is. The Strict source routing option gives the complete path from source to destination as a sequence of IP addresses. The Loose source routing option requires the packet to traverse the list of routers specified, in the order specified, but it is allowed to pass through other routers on the way. The Record route option tells each router along the path to append its IP address to the Options field. Timestamp option is like the Record

route option, except that in addition to recording its 32-bit IP address, each router also records a 32-bit timestamp.

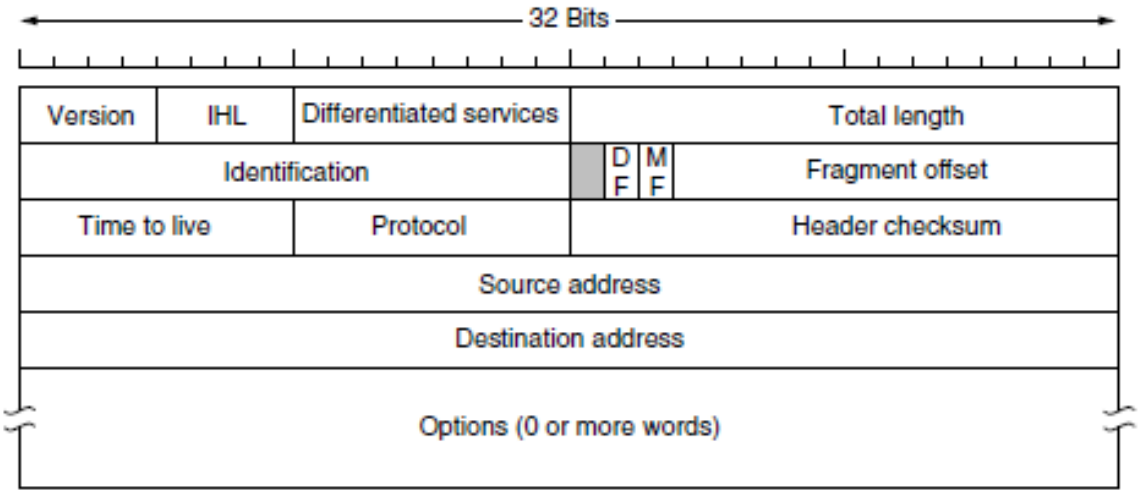


Figure 5-46. The IPv4 (Internet Protocol) header.

Part D **(2x9=18)**

(Answer any two. Each caries 9 marks)

12. Discuss about the techniques to improve QoS in internetworking.

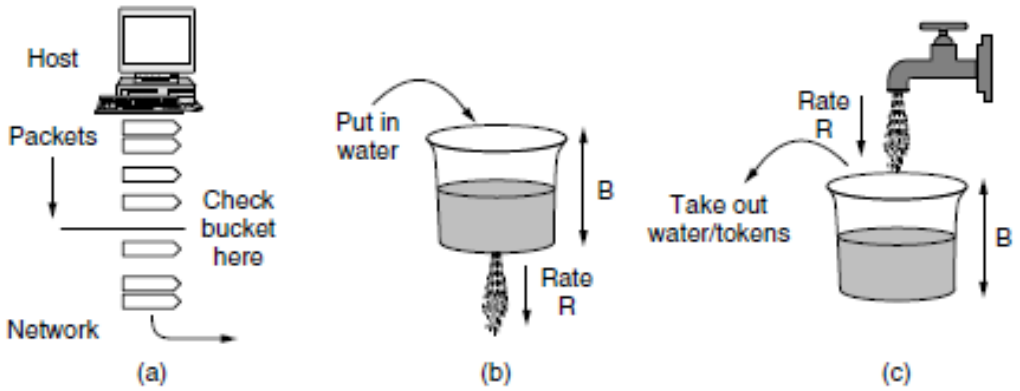
Quality of service mechanisms let a network with less capacity meet application requirements just as well at a lower cost. A stream of packets from a source to a destination is called a **flow** . A flow might be all the packets of a connection in a connection-oriented network, or all the packets sent from one process to another process in a connectionless network. The needs of each flow can be characterized by four primary parameters: bandwidth, delay, jitter, and loss. Together, these determine the **QoS** (**Quality of Service**) the flow requires. To accommodate a variety of applications, networks may support different categories of QoS. flow of data that enters the network.

Application	Bandwidth	Delay	Jitter	Loss
Email	Low	Low	Low	Medium
File sharing	High	Low	Low	Medium
Web access	Medium	Medium	Low	Medium
Remote login	Low	Medium	Medium	Medium
Audio on demand	Low	Low	High	Low
Video on demand	High	Low	High	Low
Telephony	Low	High	High	Low
Videoconferencing	High	High	High	Low

The goal is to allow applications to transmit a wide variety of traffic that suits their needs, including some bursts, yet have a simple and useful way to describe the possible traffic patterns to the network. When a flow is set up, the user and the network agree on a certain traffic pattern (i.e., shape) for that flow. In effect, the customer says to the provider ‘‘My transmission pattern will look like this; can you handle it?’’ Sometimes this agreement is called an **SLA** (**Service Level Agreement**), especially when it is made over aggregate flows and long periods of time, such as all of the traffic for a given customer. As long as the customer fulfills her part of the bargain and only sends packets according to the agreed-on contract, the provider promises to deliver them all in a timely fashion. Traffic shaping reduces congestion and thus helps the network live up to its promise.

Leaky and Token Buckets is a bucket with a small hole in the bottom, as illustrated in Fig. No matter the rate at which water enters the bucket, the outflow is at a constant rate, R, when there is any water in the bucket and zero when the bucket is empty. Also, once the bucket is full to capacity B, any additional water entering it spills over the sides and is lost.
This bucket can be used to shape or police packets entering the network, as shown in Fig. Conceptually, each host is connected to the network by an interface containing a leaky bucket. To send a packet into the network, it must be possible to put more water into the bucket. If a packet arrives when the bucket is full, the packet must either be queued until enough water leaks out to hold it or be discarded. The former might happen at a host shaping its traffic for the network as part of the operating system. The latter might happen in hardware at a provider network interface that is policing traffic entering the network. This technique was proposed by Turner and is called the **leaky bucket algorithm**.

A different but equivalent formulation is to imagine the network interface as a bucket that is being filled, as shown in Fig. 5-28(c). The tap is running at rate R and the bucket has a capacity of B, as before. Now, to send a packet we must be able to take water, or tokens, as the contents are commonly called, out of the bucket (rather than putting water into the bucket). No more than a fixed number of tokens, B, can accumulate in the bucket, and if the bucket is empty, we must wait until more tokens arrive before we can send another packet. This algorithm is called the **token bucket algorithm**.



Leaky and token buckets limit the long-term rate of a flow but allow short term bursts up to a maximum regulated length to pass through unaltered and without suffering any artificial delays. Large bursts will be smoothed by a leaky bucket traffic shaper to reduce congestion in the network. Using all of these buckets can be a bit tricky. When token buckets are used for

traffic shaping at hosts, packets are queued and delayed until the buckets permit them to be sent. When token buckets are used for traffic policing at routers in the network, the algorithm is simulated to make sure that no more packets are sent than permitted.

Packet Scheduling

Algorithms that allocate router resources among the packets of a flow and between competing flows are called **packet scheduling algorithms**. Three different kinds of resources can potentially be reserved for different flows:

1. Bandwidth.
2. Buffer space.
3. CPU cycles.

Packet scheduling algorithms allocate bandwidth and other router resources by determining which of the buffered packets to send on the output line next. We already described the most straightforward scheduler when explaining how routers work. Each router buffers packets in a queue for each output line until they can be sent, and they are sent in the same order that they arrived. This algorithm s known as **FIFO** (**First-In First-Out**), or equivalently **FCFS** (**First-Come First-Serve**). FIFO routers usually drop newly arriving packets when the queue is full. Since the newly arrived packet would have been placed at the end of the queue, this behavior is called **tail drop**. It is intuitive, and you may be wondering what alternatives exist. RED algorithm chose a newly arriving packet to drop at random when the average queue length grew large.

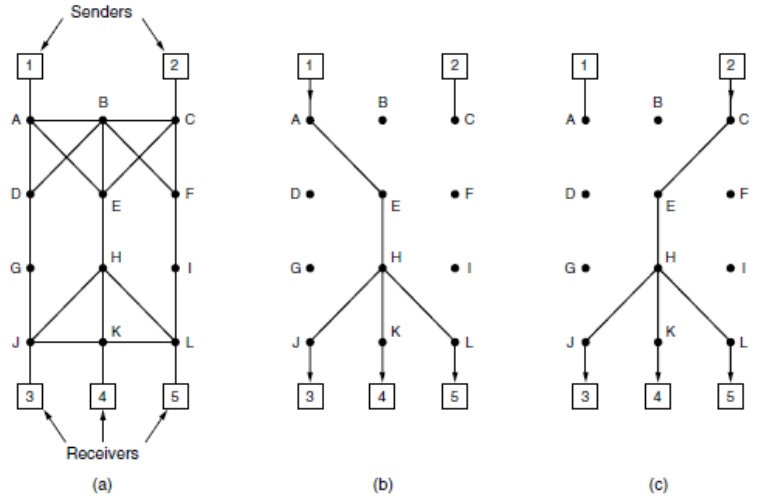
Admission Control

The user offers a flow with an accompanying QoS requirement to the network. The network then decides whether to accept or reject the flow based on its capacity and the commitments it has made to other flows. If it accepts, the network reserves capacity in advance at routers to guarantee QoS when traffic is sent on the new flow. The reservations must be made at all of the routers along the route that the packets take through the network. Any routers on the path without reservations might become congested, and a single congested router can break the QoS guarantee.

Integrated Services

RSVP—The Resource reSerVation Protocol

The main part of the integrated services architecture that is visible to the users of the network is **RSVP**. It is described in RFCs 2205–2210. This protocol is used for making the reservations; other protocols are used for sending the data. RSVP allows multiple senders to transmit to multiple groups of receivers, permits individual receivers to switch channels freely, and optimizes bandwidth use while at the same time eliminating congestion. In its simplest form, the protocol uses multicast routing using spanning trees. Each group is assigned a group address. To send to a group, a sender puts the group's address in its packets. The standard multicast routing algorithm then builds a spanning tree covering all group members. The routing algorithm is not part of RSVP. The only difference from normal multicasting is a little extra information that is multicast to the group periodically to tell the routers along the tree to maintain certain data structures in their memories.

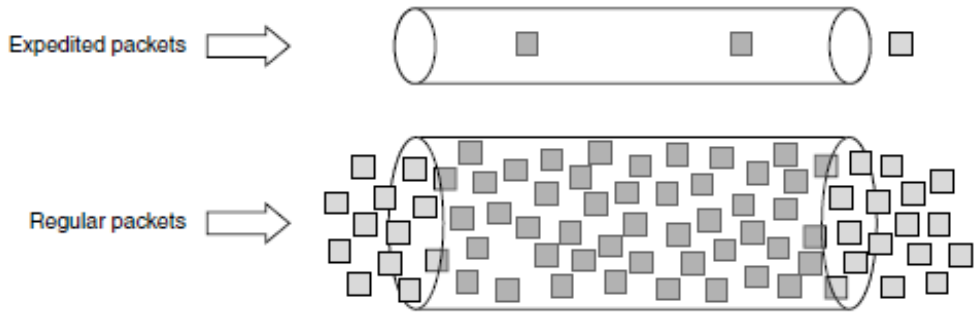


Differentiated Services

Differentiated services can be offered by a set of routers forming an administrative domain . The administration defines a set of service classes with corresponding forwarding rules. If a customer subscribes to differentiated services, customer packets entering the domain are marked with the class to which they belong. This information is carried in the Differentiated services field of IPv4 and IPv6 packets. The classes are defined as **per hop behaviors** because they correspond to the treatment the packet will receive at each router, not a guarantee across the network. Better service is provided to packets with some per-hop behaviors (e.g., premium service) than to others (e.g., regular service). Traffic within a class may be required to conform to some specific shape, such as a leaky bucket with some specified drain rate.

Expedited Forwarding

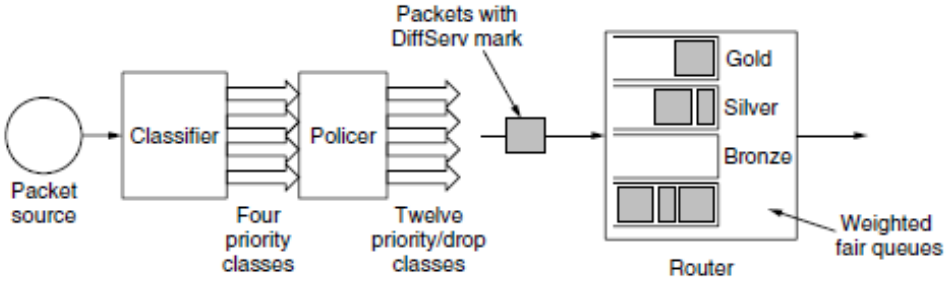
The idea behind expedited forwarding is very simple. Two classes of service are available: regular and expedited. The vast majority of the traffic is expected to be regular, but a limited fraction of the packets are expedited. The expedited packets should be able to transit the network as though no other packets were present. In this way they will get low loss, low delay and low jitter service.



Assured Forwarding

Assured forwarding specifies that there shall be four priority classes, each class having its own resources. The top three classes might be called gold, silver, and bronze. In addition, it defines three discard classes for packets that are experiencing congestion: low, medium,

and high. Taken together, these two factors define 12 service classes. Figure shows one way packets might be processed under assured forwarding. The first step is to classify the packets into one of the four priority classes. As before, this step might be done on the sending host or in the ingress router, and the rate of higher-priority packets may be limited by the operator as part of the service offering.The next step is to determine the discard class for each packet. This is done by passing the packets of each priority class through a traffic policer such as a token bucket. The policer lets all of the traffic through, but it identifies packets that fit within small bursts as low discard, packets that exceed small bursts as medium discard, and packets that exceed large bursts as high discard. The combination of priority and discard class is then encoded in each packet. Finally, the packets are processed by routers in the network with a packet scheduler that distinguishes the different classes.



13. Illustrate the routing procedure in mobile networks.

Design goals to be considered for mobile IP are

- Each mobile host must be able to use its home IP address anywhere.
- Software changes to the fixed hosts were not permitted.
- Changes to the router software and tables were not permitted.
- Most packets for mobile hosts should not make detours on the way.
- No overhead should be incurred when a mobile host is at home.

Mobile Node (MN)

• system (node) that can change the point of connection to the network without changing its IP address

• Home Agent (HA)

• system in the home network of the MN, typically a router

• registers the location of the MN, tunnels IP datagrams to the COA

• Foreign Agent (FA)

• system in the current foreign network of the MN, typically a router

• forwards the tunneled datagrams to the MN, typically also the default router for the MN

• Care-of Address (COA)

• address of the current tunnel end-point for the MN (at FA or MN)

• actual location of the MN from an IP point of view can be chosen, e.g., via DHCP

• Correspondent Node (CN)

• communication partner

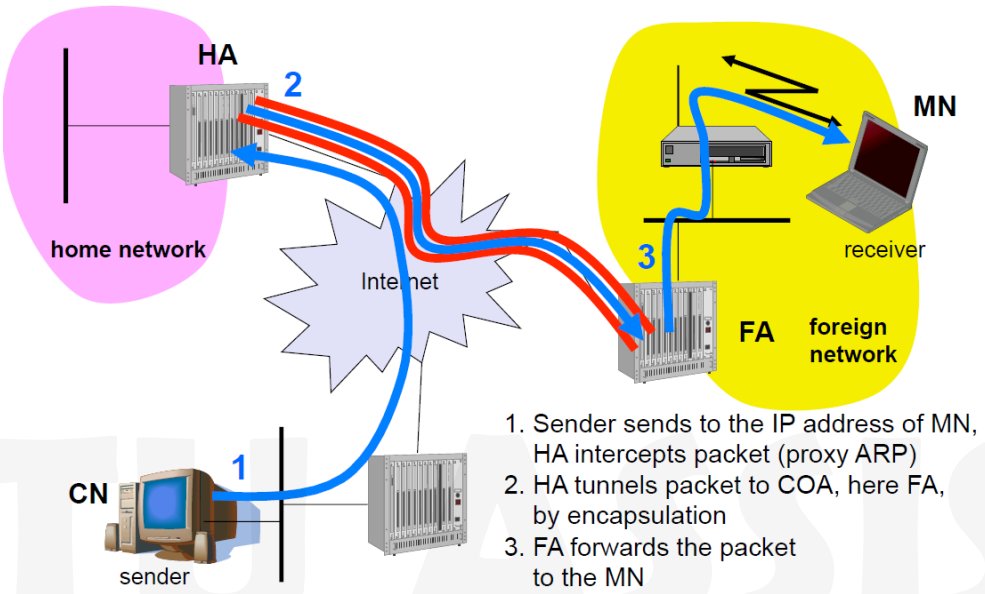
Agent Advertisement

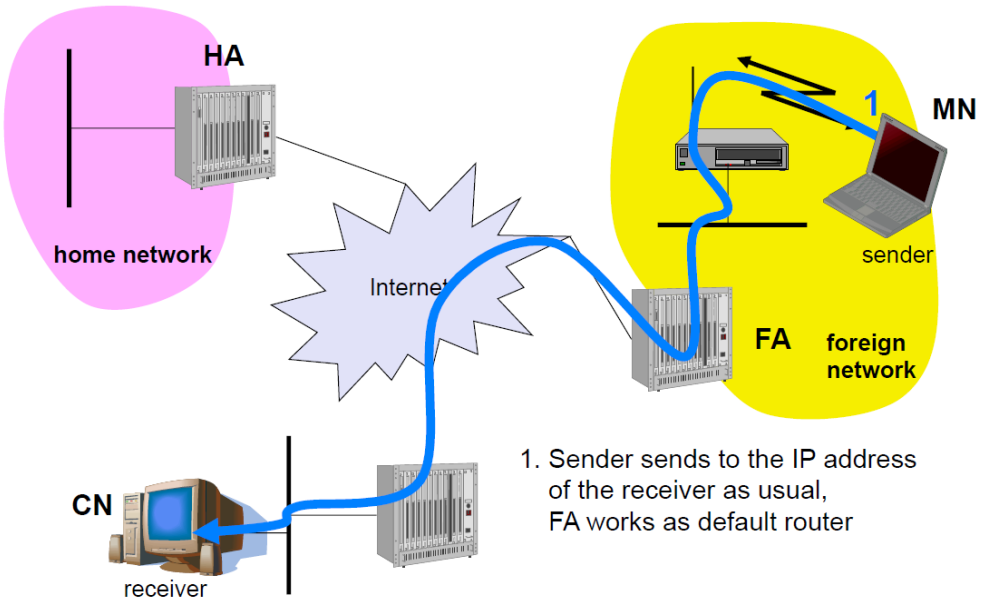
• HA and FA periodically send advertisement messages into their physical subnets

- MN listens to these messages and detects, if it is in the home or a foreign network
- MN reads a COA from the FA advertisement messages
- Registration (always limited lifetime!)
- MN signals COA to the HA via the FA, HA acknowledges via FA to MN
- these actions have to be secured by authentication

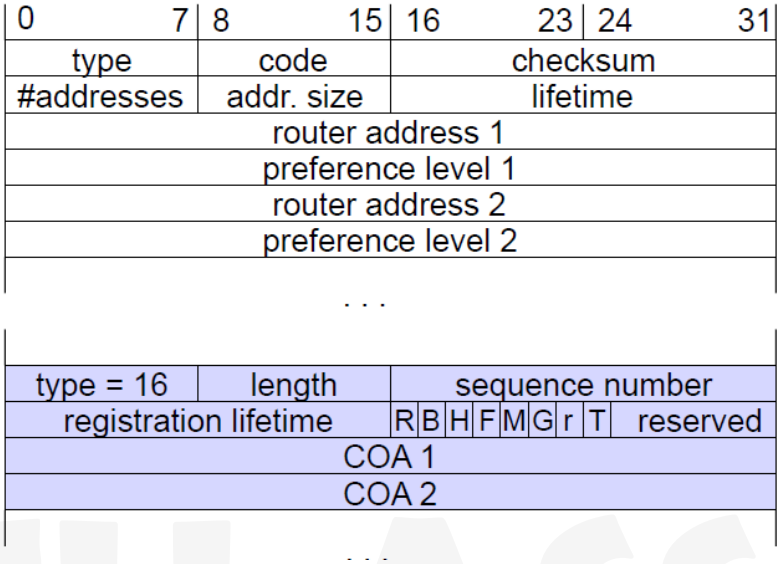
• Advertisement

- HA advertises the IP address of the MN i.e. standard routing information
- routers adjust their entries, these are stable for a longer time (HA responsible for a MN over a longer period of time)
- packets to the MN are sent to the HA,• independent of changes in COA/FA

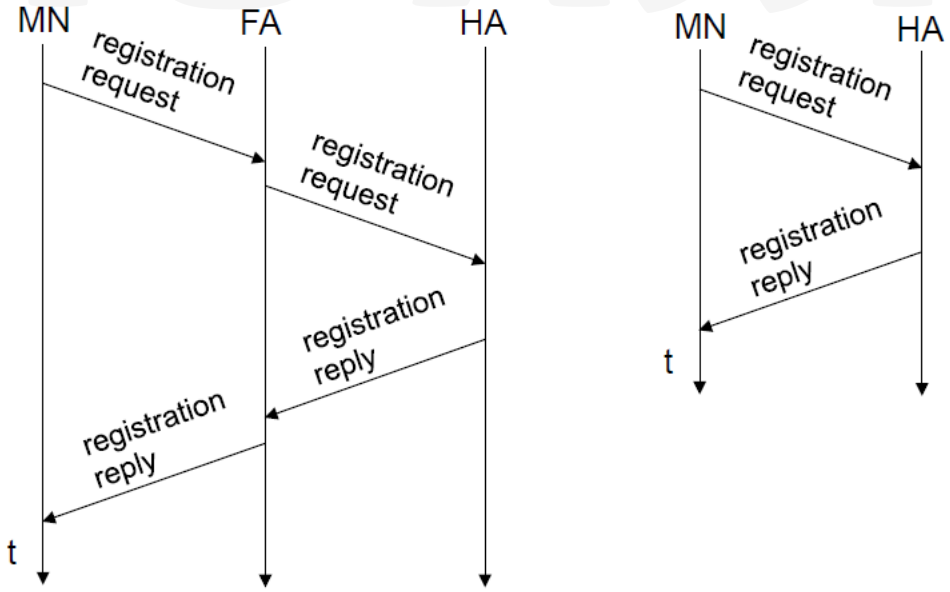




Agent Advertisment Frame Structure



Registration Process



Encapsulation Frame Structure

ver.	IHL	DS (TOS)	length
IP identification		flags	fragment offset
TTL	*IP-in-IP*	IP checksum	
IP address of HA			
Care-of address COA			
ver.	IHL	DS (TOS)	length
IP identification		flags	fragment offset
TTL	lay. 4 prot.	IP checksum	
IP address of CN			
IP address of MN			
TCP/UDP/ ... payload			

14. Explain the various congestion control techniques.

The presence of congestion means that the load is (temporarily) greater than the resources (in a part of the network) can handle. Two solutions come to mind: increase the resources or decrease the load.

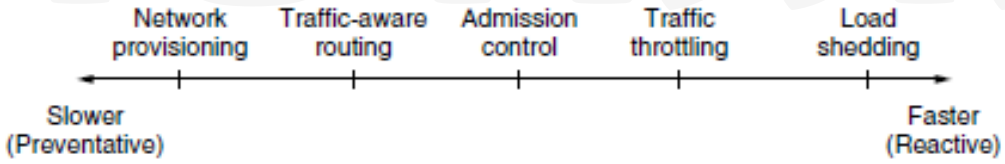


Figure 5-22. Timescales of approaches to congestion control.

Traffic-Aware Routing

The first approach we will examine is traffic-aware routing. The routing schemes we looked at in Sec 5.2 used fixed link weights. These schemes adapted to changes in topology, but not to changes in load. The goal in taking load into account when computing routes is to shift traffic away from hotspots that will be the first places in the network to experience congestion. The most direct way to do this is to set the link weight to be a function of the (fixed) link bandwidth and propagation delay plus the (variable) measured load or average queuing delay. Least-weight paths will then favor paths that are more lightly loaded, all else being equal. Internet routing protocols do not generally adjust their routes depending on the load. Instead, adjustments are made outside the routing protocol by slowly changing its inputs. This is called **traffic engineering**.

Admission Control

One technique that is widely used in virtual-circuit networks to keep congestion at bay is **admission control**. The idea is simple: do not set up a new virtual circuit unless the network can carry the added traffic without becoming congested. Thus, attempts to set up a virtual circuit may fail. This is better than the alternative, as letting more people in when the network is busy just makes matters worse. By analogy, in the telephone system, when a switch gets overloaded it practices admission control by not giving dial tones. Admission control can also be combined with traffic-aware routing by considering routes around traffic hotspots as part of the setup procedure.

Traffic Throttling

In the Internet and many other computer networks, senders adjust their transmissions to send as much traffic as the network can readily deliver. In this setting, the network aims to operate just before the onset of congestion. When congestion is imminent, it must tell the senders to throttle back their transmissions and slow down. This feedback is business as usual rather than an exceptional situation. The term **congestion avoidance** is sometimes used to contrast this operating point with the one in which the network has become (overly) congested. both datagram networks and virtual-circuit networks. Each approach must solve two problems. First, routers must determine when congestion is approaching, ideally before it has arrived. To do so, each router can continuously monitor the resources it is using. Three possibilities are the utilization of the output links, the buffering of queued packets inside the router, and the number of packets that are lost due to insufficient buffering. The second problem is that routers must deliver timely feedback to the senders that are causing the congestion. Congestion is experienced in the network, but relieving congestion requires action on behalf of the senders that are using the network. To deliver feedback, the router must identify the appropriate senders. It must then warn them carefully, without sending many more packets into the already congested network. Different schemes use different feedback mechanisms.

Choke Packets

The most direct way to notify a sender of congestion is to tell it directly. In this approach, the router selects a congested packet and sends a **choke packet** back to the source host, giving it the destination found in the packet. The original packet may be tagged (a header bit is turned on) so that it will not generate any more choke packets farther along the path and then forwarded in the usual way. To avoid increasing load on the network during a time of congestion, the router may only send choke packets at a low rate.

When the source host gets the choke packet, it is required to reduce the traffic sent to the specified destination, for example, by 50%. In a datagram network, imply picking packets at random when there is congestion is likely to cause choke packets to be sent to fast senders, because they will have the most packets in the queue. The feedback implicit in this protocol can help prevent congestion yet not throttle any sender unless it causes trouble. For the same reason, it is likely that multiple choke packets will be sent to a given host and destination. The host should ignore these additional chokes for the fixed time interval until its reduction in traffic takes effect. After that period, further choke packets indicate that the network is still congested.

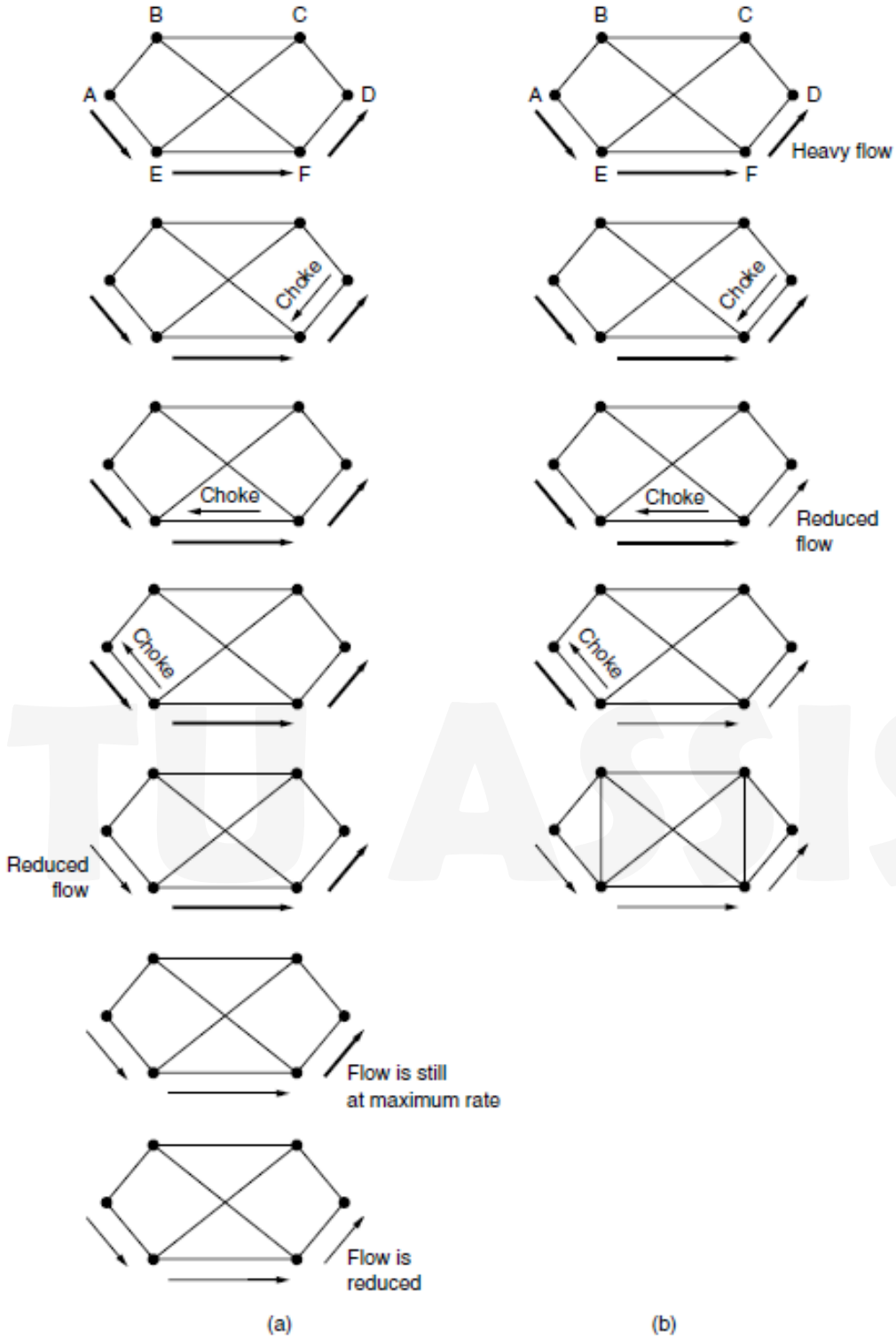


Figure 5-26. (a) A choke packet that affects only the source. (b) A choke packet that affects each hop it passes through.

Explicit Congestion Notification

Instead of generating additional packets to warn of congestion, a router can tag any packet it forwards (by setting a bit in the packet's header) to signal that it is experiencing congestion. When the network delivers the packet, the destination can note that there is congestion and inform the sender when it sends a reply packet. The sender can then throttle its transmissions as before. This design is called **ECN** (**Explicit Congestion Notification**) and is used in the Internet.

Load shedding

It is a fancy way of saying that when routers are being inundated by packets that they cannot handle, they just throw them away. The term comes from the world of electrical power generation, where it refers to the practice of utilities intentionally blacking out certain areas to save the entire grid from collapsing on hot summer days when the demand for electricity greatly exceeds the supply. The key question for a router drowning in packets is which packets to drop. The preferred choice may depend on the type of applications that use the network.

Random Early Detection

By having routers drop packets early, before the situation has become hopeless, there is time for the source to take action before it is too late. A popular algorithm for doing this is called **RED** (**Random Early Detection**) . To determine when to start discarding, routers maintain a running average of their queue lengths. When the average queue length on some link exceeds a threshold, the link is said to be congested and a small fraction of the packets are dropped at random. Picking packets at random makes it more likely that the fastest senders will see a packet drop; this is the best option since the router cannot tell which source is causing the most trouble in a datagram network. The affected sender will notice the loss when there is no acknowledgement, and then the transport protocol will slow down. The lost packet is thus delivering the same message as a choke packet, but implicitly, without the router sending any explicit signal. RED routers improve performance compared to routers that drop packets only when their buffers are full, though they may require tuning to work well.

15. Explain Link State routing algorithm.

The idea behind link state routing is fairly simple and can be stated as five parts. Each router must do the following things to make it work:

1. Discover its neighbors and learn their network addresses.
2. Set the distance or cost metric to each of its neighbors.
3. Construct a packet telling all it has just learned.
4. Send this packet to and receive packets from all other routers.
5. Compute the shortest path to every other router.

In effect, the complete topology is distributed to every router. Then Dijkstra's algorithm can be run at each router to find the shortest path to every other router.

Learning about the Neighbors

When a router is booted, its first task is to learn who its neighbors are. It accomplishes this goal by sending a special HELLO packet on each point-to-point line. The router on the other end is expected to send back a reply giving its name. These names must be globally unique because when a distant router later hears that three routers are all connected to F, it is essential that it can determine whether all three mean the same F.

Setting Link Costs

The link state routing algorithm requires each link to have a distance or cost metric for finding shortest paths. The cost to reach neighbors can be set automatically, or configured by the network operator. If the network is geographically spread out, the delay of the links may be factored into the cost so that paths over shorter links are better choices. The most direct way to determine this delay is to send over the line a special ECHO packet that the other side is required to send back immediately. By measuring the round-trip time and dividing it by two, the sending router can get a reasonable estimate of the delay.

Building Link State Packets

Once the information needed for the exchange has been collected, the next step is for each router to build a packet containing all the data. The packet starts with the identity of the sender, followed by a sequence number and a list of neighbors. The cost to each neighbor is also given.

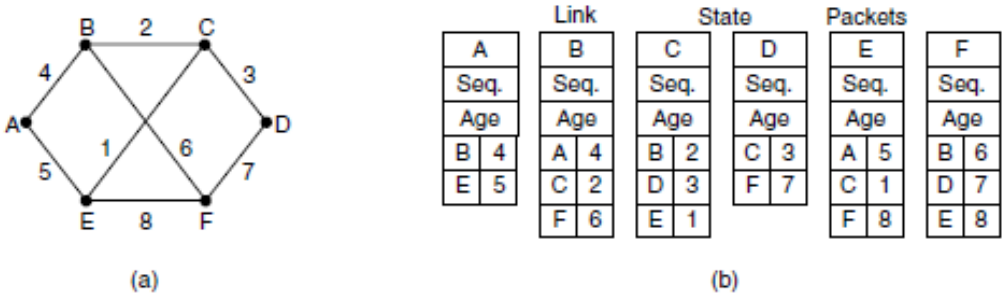


Figure 5-12. (a) A network. (b) The link state packets for this network.

Distributing the Link State Packets

The fundamental idea is to use flooding to distribute the link state packets to all routers. To keep the flood in check, each packet contains a sequence number that is incremented for each new packet sent. Routers keep track of all the (source router, sequence) pairs they see. When a new link state packet comes in, it is checked against the list of packets already seen. If it is new, it is forwarded on all lines except the one it arrived on. If it is a duplicate, it is discarded. If a packet with a sequence number lower than the highest one seen so far ever arrives, it is rejected as being obsolete as the router has more recent data.

Computing the New Routes

Once a router has accumulated a full set of link state packets, it can construct the entire network graph because every link is represented. Every link is, in fact, represented twice, once for each direction. The different directions may even have different costs. The shortest-path computations may then find different paths from router A to B than from router B to A.

Now Dijkstra's algorithm can be run locally to construct the shortest paths to all possible destinations. The results of this algorithm tell the router which link to use to reach each destination. This information is installed in the routing tables, and normal operation is resumed.

Compared to distance vector routing, link state routing requires more memory and computation. For a network with n routers, each of which has k neighbors, the memory required to store the input data is proportional to kn, which is at least as large as a routing table listing all the destinations. Also, the computation time grows faster than kn, even with the most efficient data structures, an issue in large networks.

Part E **(4x10=40)**

(Answer any four. Each caries 10 marks)

16. Illustrate the working of OSPF protocol.

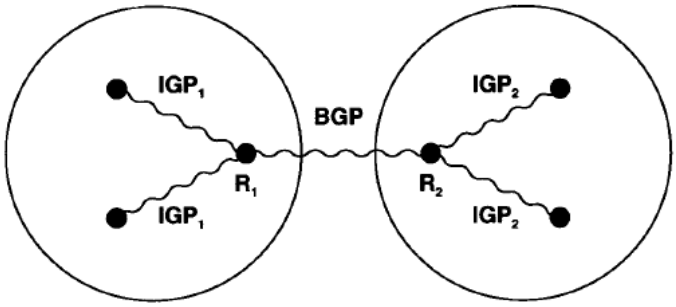


Figure 16.3 Conceptual view of two autonomous systems each using its own IGP internally, but using BGP to communicate between an exterior router and the other system.

- Interior Gateway Protocol (IGP) as a generic description that refers to any algorithm that interior routers use when they exchange network reachability and routing information. Once the reachability information for an entire autonomous system has been assembled, one of the routers in the system can advertise it to other autonomous systems using an Exterior Gateway Protocol.
- Link state routing algorithm, which uses SPF to compute shortest paths, scales better than a distance-vector algorithm. IETF has designed an IGP that uses the link state algorithm called Open SPF (OSPF), the new protocol tackles several ambitious goals.

1. Making it an open standard that anyone can implement without paying license fees has encouraged many vendors to support OSPF.
2. OSPF includes type of service routing.
3. OSPF provides load balancing.
4. To permit growth and make the networks at a site easier to manage, OSPF allows a site to partition its networks and routers into subsets called areas. Each area is self-contained; knowledge of an area's topology remains hidden from other areas.
5. The OSPF protocol specifies that all exchanges between routers can be authenticated.
6. OSPF includes support for host-specific, subnet-specific, and classless routes as well as classful network-specific routes.
7. To accommodate multi-access networks like Ethernet, OSPF extends the SPF algorithm
8. To permit maximum flexibility, OSPF allows managers to describe a virtual network topology that abstracts away from the details of physical connections.
9. OSPF allows routers to exchange routing information learned from other (external) sites.

OSPF Message Format

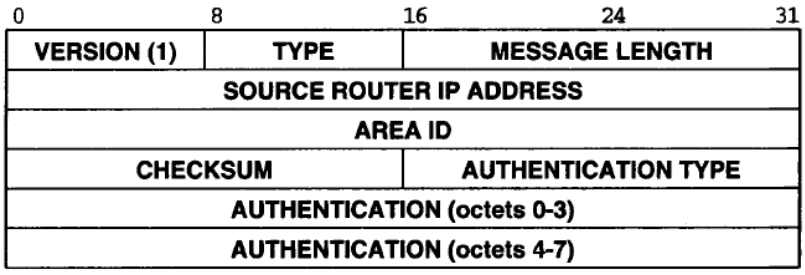


Figure 16.7 The fixed 24-octet OSPF message header.

Type	Meaning
1	Hello (used to test reachability)
2	Database description (topology)
3	Link status request
4	Link status update
5	Link status acknowledgement

- VERSION specifies the version of the protocol.
- TYPE identifies the message type
- SOURCE ROUTER IP ADDRESS gives the address of the sender
- AREA ID gives the 32-bit identification number for the area.
- AUTHENTICATION TYPE specifies which authentication scheme is used (currently, 0 means no authentication and I means a simple password is used).

OSPF Hello Message Format

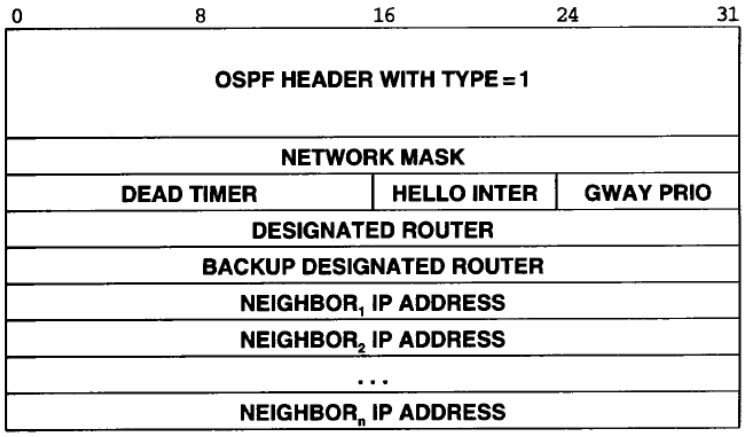


Figure 16.8 OSPF *hello* message format. A pair of neighbor routers exchanges these messages periodically to test reachability.

- NETWORK MASK contains a mask for the network over which the message
- has been sent
- DEAD TIMER gives a time in seconds after which a nonresponding neighbor is considered dead.
- HELLOPO INTER is the normal period, in seconds, between hello messages.
- GWAYPRIO is the integer priority of this router, and is used in selecting a backup designated router.
- DESIGNATED ROUTER and BACKUP DESIGNATEDR OUTER contain IP addresses that give the sender's view of the designated router andbackup designated router for the network over which the message is sent.
- NEIGHBOR, IP ADDRESS give the IP addresses of all neighbors from which the sender has recently received hello messages.

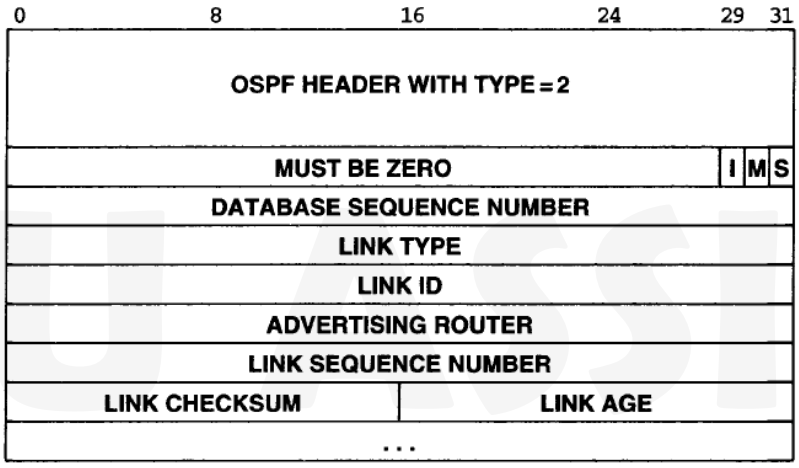


Figure 16.9 OSPF *database description* message format. The fields starting at *LINK TYPE* are repeated for each link being specified.

Link Type	Meaning
1	Router link
2	Network link
3	Summary link (IP network)
4	Summary link (link to border router)
5	External link (link to another site)

- LINK TYPE through LINK AGE describe one link in the network topology; they are repeated for each link.
- LINK ID gives an identification for the link (IP address of a router or network)
- ADVERTISING ROUTER specifies the address of the router advertising this link
- LINK SEQUENCE NUMBER contains an integer generated by that router to ensure that messages are not missed or received out of order.

- LINK CHECKSUM provides further assurance that the link information has not been corrupted.
- LINK AGE also helps order messages it gives the time in seconds since the link was established.

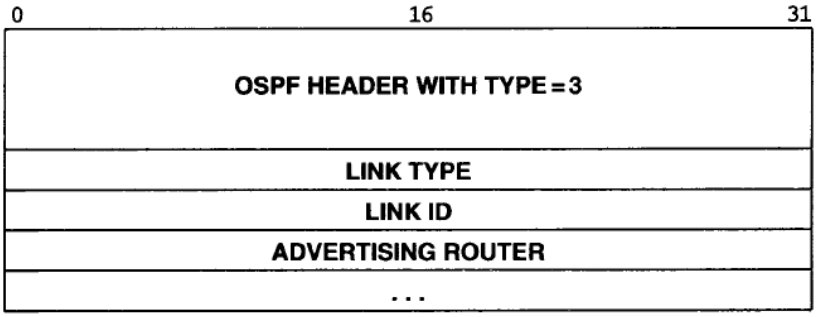


Figure 16.10 OSPF *link status request* message format. A router sends this message to a neighbor to request current information about a specific set of links.

- After exchanging database description messages with a neighbor, a router may discover that parts of its database are out of date.
- To request that the neighbor supply updated information, the router sends a link status request message.
- The neighbor responds with the most current information it has about those links.
- More than one request message may be needed if the list of requests is long.

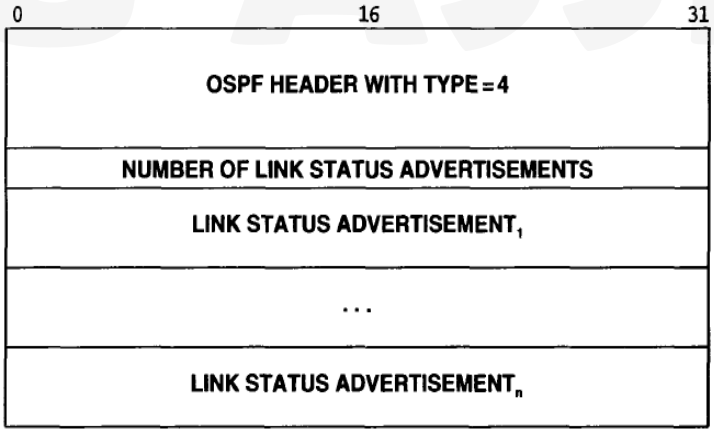


Figure 16.11 OSPF *link status update* message format. A router sends such a message to broadcast information about its directly connected links to all other routers.

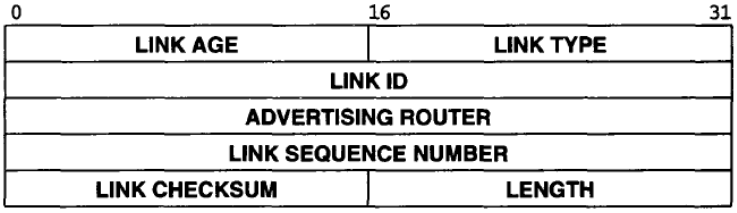


Figure 16.12 The format of the header used for all link status advertisements.

- Routers broadcast the status of links with a link status update message.
- LINK TYPE field in the link status header specifies which of the formats has been used. Thus, a router that receives a link status update message knows exactly which of the described destinations lie inside the site and which are external.

17. Explain how control messages are sent using ICMP protocol.

- To allow routers in an internet to report errors or provide information about unexpected circumstances, Internet Control Message Protocol (ICMP), must be included in every IP implementation.
- ICMP messages travel across the internet in the data portion of IP datagrams.
- The ultimate destination of an ICMP message is not an application program or user on the destination machine but the Internet Protocol software on that machine.
- ICMP error message arrives, the ICMP software module handles it.
- When a datagram causes an error, ICMP can only report the error condition back to the original source of the datagram; the source must relate the error to an individual program or take other action to correct the problem.

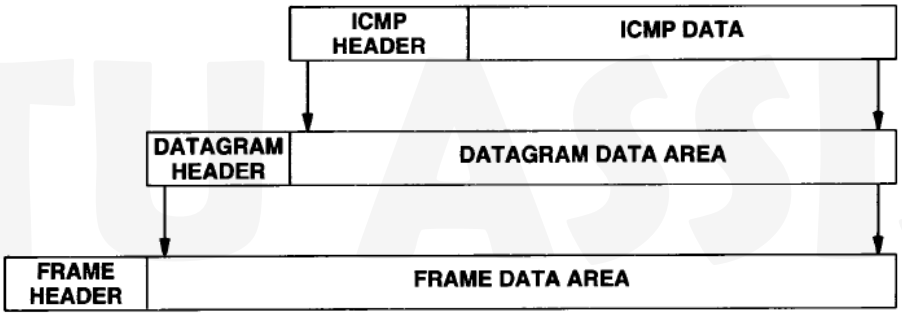


Figure 9.1 Two levels of ICMP encapsulation. The ICMP message is encapsulated in an IP datagram, which is further encapsulated in a frame for transmission. To identify ICMP, the datagram protocol field contains the value *1*.

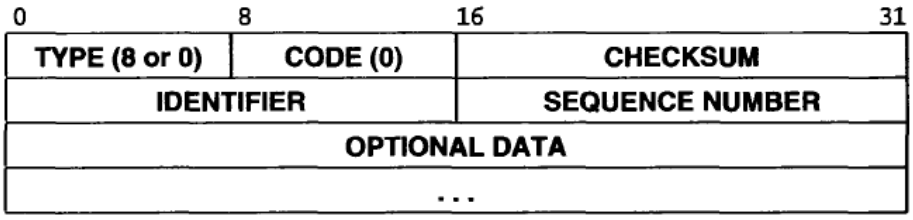


Figure 9.2 ICMP echo request or reply message format.

Type Field	ICMP Message Type
0	Echo Reply
3	Destination Unreachable
4	Source Quench
5	Redirect (change a route)
8	Echo Request
9	Router Advertisement
10	Router Solicitation
11	Time Exceeded for a Datagram
12	Parameter Problem on a Datagram
13	Timestamp Request
14	Timestamp Reply
15	Information Request (obsolete)
16	Information Reply (obsolete)
17	Address Mask Request
18	Address Mask Reply

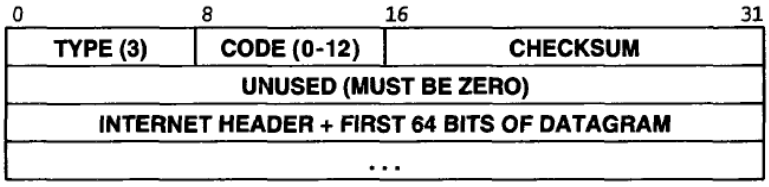


Figure 9.3 ICMP destination unreachable message format.

- Whenever an error prevents a router from routing or delivering a datagram, the router sends a destination unreachable message back to the source and then **drops (i.e., discards) the datagram.**
- **Network unreachable errors usually imply routing** failures
- **Host unreachable** errors imply delivery failure.
- Destinations may be unreachable because hardware is temporarily out of service, because the sender specified a nonexistent destination address, or because the router does not have a route to the destination network.

Code Value	Meaning
0	Network unreachable
1	Host unreachable
2	Protocol unreachable
3	Port unreachable
4	Fragmentation needed and DF set
5	Source route failed
6	Destination network unknown
7	Destination host unknown
8	Source host isolated
9	Communication with destination network administratively prohibited
10	Communication with destination host administratively prohibited
11	Network unreachable for type of service
12	Host unreachable for type of service

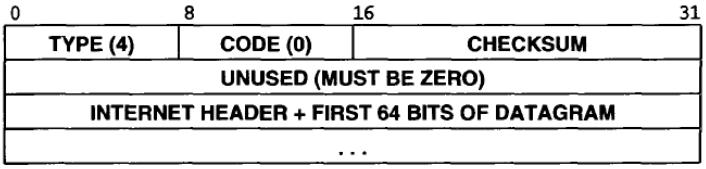


Figure 9.4 ICMP source quench message format. A congested router sends one source quench message each time it discards a datagram; the datagram prefix identifies the datagram that was dropped.

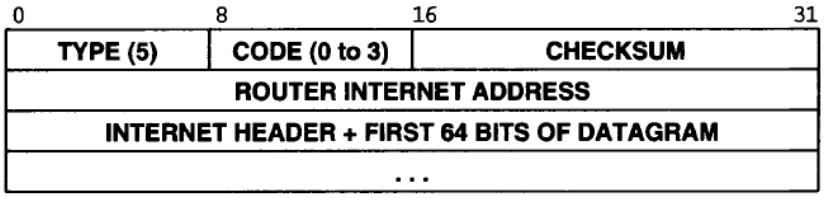


Figure 9.6 ICMP redirect message format.

- ROUTER INTERNET ADDRESS field contains the address of a router that the host is to use to reach the destination mentioned in the datagram header.
- The INTERNETHEADER field contains the IP header plus the next 64 bits of the datagram that triggered the message. Thus, a host receiving an ICMP redirect examines the datagram prefix to determine the datagram's destination address.
- The CODE field of an ICMP redirect message further specifies how to interpret the destination address, based on values assigned as follows:

Code Value	Meaning
0	Redirect datagrams for the Net (now obsolete)
1	Redirect datagrams for the Host
2	Redirect datagrams for the Type of Service† and Net
3	Redirect datagrams for the Type of Service and Host

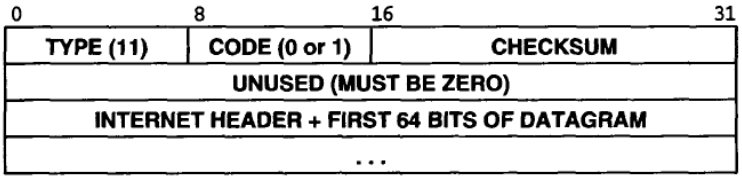


Figure 9.7 ICMP time exceeded message format. A router sends this message whenever a datagram is discarded because the time-to-live field in the datagram header has reached zero or because its reassembly timer expired while waiting for fragments.

- To prevent datagrams from circling forever in a TCP/IP internet, each IP datagram contains a time-to-live counter, sometimes called a hop count.
- A router decrements the time-to-live counter whenever it processes the datagram and discards the datagram when the count reaches zero.
- Whenever a router discards a datagram because its hop count has reached zero or because a timeout occurred while waiting for fragments of a datagram, it sends an ICMP time exceeded message back to the datagram's source.

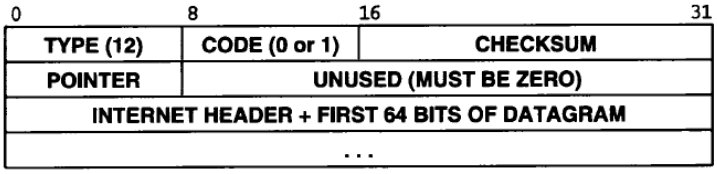


Figure 9.8 ICMP parameter problem message format. Such messages are only sent when the problem causes the datagram to be dropped.

When a router or host finds problems with a datagram not covered by previous ICMP error messages, it sends a parameter problem message to the original source. One possible cause of such problems occurs when arguments to an option are incorrect. It is only sent when the problem is so severe that the datagram must be discarded. Sender uses the POINTER field in the message header to identify the octet in the datagram that caused the problem. Code 1 is used to report that a required option is missing . POINTER field is not used for code 1.

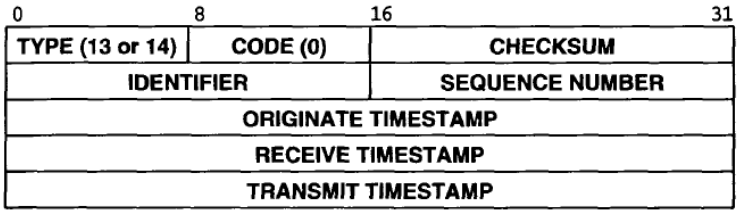


Figure 9.9 ICMP timestamp request or reply message format.

ICMP message to obtain the time from another machine and used to synchronize clocks . A requesting machine sends an ICMP timestamp request message to another machine, asking that the second machine return its current value for the time of day. The receiving machine returns a timestamp reply back to the machine making the request. TYPE field identifies the message as a request (13) or a reply (14); the IDENTIFIER and SEQUENCE NUMBER fields are used by the source to associate replies with requests. Remaining fields specify times, given in milliseconds since midnight, Universal Time. The ORIGINATE TIMESTAMP field is filled in by the original sender just before the packet is transmitted, the RECEIVE TIMESTAMP field is filled immediately upon receipt of a request, and the TRANSMIT TIMESTAMP field is filled immediately before the reply is transmitted. Hosts use the three timestamp fields to compute estimates of the delay time between them and to synchronize their clocks. Because the reply includes the ORIGINATE TIMESTAMP field, a host can compute the total time required for a request to travel to a destination, because the reply carries both the time at which the request entered the remote machine, as well as the time at which the reply left, the host can compute the network transit time, and from that, estimate the differences in remote and local clocks. The ICMP information request and information reply messages (types 15 and 16) are now considered obsolete and should not be used. They were originally intended to allow hosts to discover their internet address at system startup.

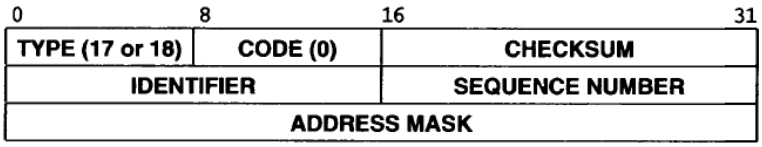


Figure 9.10 ICMP address mask request or reply message format. Usually, hosts broadcast a request without knowing which specific router will respond.

To participate in subnet addressing, a host needs to know which bits of the 32-bit internet address correspond to the physical network and which correspond to host identifiers. The information needed to interpret the address is represented in a 32-bit quantity called the subnet mask. To learn the subnet mask used for the local network, a machine can send an address mask request message to a router and receive an address mask reply. The machine making the request can either send the message directly, if it knows the router's address, or broadcast the message if it does not. The ICMP router discovery scheme helps in two ways. First, instead of providing a statically configured router address via a bootstrap protocol, the scheme allows a host to obtain information directly from the router itself. Second, the mechanism uses a softstate technique with timers to prevent hosts from retaining a route after a router crashes. Routers advertise their information periodically, and a host discards a route if the timer for a route expires. Besides the TYPE, CODE, and CHECKSUM fields, the message contains a field NUM ADDRS that specifies the number of address entries which follow (often I), an ADDR SIZE field that specifies the size of an address in 32-bit units, and a LIFETIME field that specifies the time in seconds a host may use the advertised address(es). The default value for LIFETIME is 30 minutes, and the default value for periodic retransmission is 10 minutes, which means that a host will not discarda route if the host misses a single advertisement message.The remainder of the message consists of NUM ADDRS pairs of fields, where each pair contains a ROUTER ADDRESS and an integer PRECEDENCE LEVEL for the route. The precedence value is a two's complement integer; a host chooses the route with highest precedence.If the router and the network support multicast, a router multicast ICMP router advertisement messages to the all-systems multicast address (i.e., 224.0.0.1). If not, the router sends the messages to the limited broadcast address (i.e., the all 1's address). Of course, a host must never send a router advertisement message.

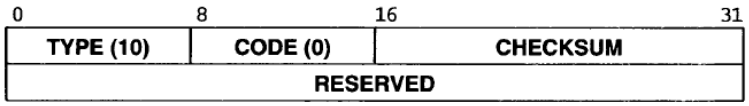


Figure 9.12 ICMP router solicitation message. A host sends a solicitation after booting to request that routers on the local net immediately respond with an ICMP router advertisement.

Although the designers provided a range of values to be used as the delay between successive router advertisements, they chose the default of 10 minutes. The value was selected as a compromise between rapid failure detection and low overhead. A smaller value would allow more rapid detection of router failure, but would increase network traffic; a larger value would decrease traffic, but would delay failure detection. One of the issues the designers considered was how to accommodate a large number of routers on the same network. From the point of view of a host, the default delay has a severe disadvantage: a host cannot afford to wait many minutes for an advertisement when it first boots. To avoid such delays, the designers included an ICMP router solicitation message that allows a host to request an immediate advertisement.

18 a. Give the format of TCP header and discuss the relevance of various fields.

TCP (**Transmission Control Protocol**) was specifically designed to provide a reliable end-to-end byte stream over an unreliable internetwork. The sending and receiving TCP entities exchange data in the form of segments. A **TCP segment** consists of a fixed 20-byte header (plus an optional part) followed by zero or more data bytes. The TCP software decides how big segments should be. It can accumulate data from several writes into one segment or
can split data from one write over multiple segments. Two limits restrict the segment size. First, each segment, including the TCP header, must fit in the 65,515- byte IP payload. Second, each link has an **MTU** (**Maximum Transfer Unit**). Each segment must fit in the MTU at the sender and receiver so that it can be sent and received in a single, unfragmented packet. In practice, the MTU is generally 1500 bytes (the Ethernet payload size) and thus defines the upper bound on segment size.

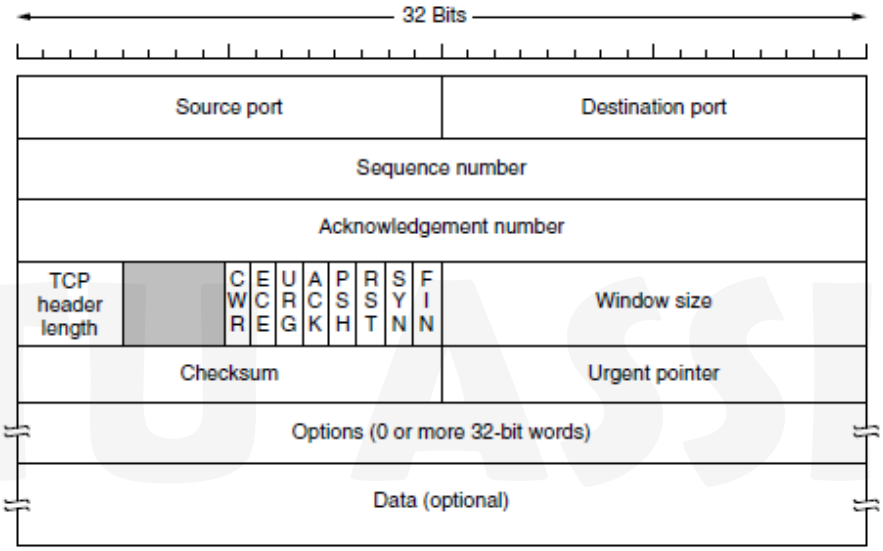


Figure 6-36. The TCP header.

The basic protocol used by TCP entities is the sliding window protocol with a dynamic window size. When a sender transmits a segment, it also starts a timer. When the segment arrives at the destination, the receiving TCP entity sends back a segment (with data if any exist, and otherwise without) bearing an acknowledgement number equal to the next sequence number it expects to receive and the remaining window size. If the sender's timer goes off before the acknowledgement is received, the sender transmits the segment again. Every segment begins with a fixed-format, 20-byte header. The fixed header may be followed by header options. After the options, if any, up to 65,495 data bytes may follow, where the first 20 refer to the IP header and the second to the TCP header. Segments without any data are legal and are commonly used for acknowledgements and control messages. The Source port and Destination port fields identify the local end points of the connection. A TCP port plus its host's IP address forms a 48-bit unique end point. The source and destination end
points together identify the connection. This connection identifier is called a **5 tuple** because it consists of five pieces of information: the protocol (TCP), source IP and source port, and destination IP and destination port. The Sequence number and Acknowledgement number fields perform their usual functions. Note that the latter specifies the next in-order byte expected, not the last byte correctly received. It is a **cumulative acknowledgement** because it

summarizes the received data with a single number. It does not go beyond lost data. Both are 32 bits because every byte of data is numbered in a TCP stream. The TCP header length tells how many 32-bit words are contained in the TCP header. This information is needed because the Options field is of variable length, so the header is, too. Technically, this field really indicates the start of the data within the segment, measured in 32-bit words, but that number is just the header length in words, so the effect is the same.

Next comes a 4-bit field that is not used. CWR and ECE are used to signal congestion when ECN (Explicit Congestion Notification) is used, as specified in RFC 3168.
ECE is set to signal an ECN-Echo to a TCP sender to tell it to slow down when the TCP receiver gets a congestion indication from the network. CWR is set to signal Congestion Window Reduced from the TCP sender to the TCP receiver so that it knows the sender has slowed down and can stop sending the ECN-Echo. URG is set to 1 if the Urgent pointer is in use. The Urgent pointer is used to indicate a byte offset from the current sequence number at which urgent data are to be found. This facility is in lieu of interrupt messages. As we mentioned above, this facility is a bare-bones way of allowing the sender to signal the receiver without getting TCP itself involved in the reason for the interrupt, but it is
seldom used.The ACK bit is set to 1 to indicate that the Acknowledgement number is valid. This is the case for nearly all packets. If ACK is 0, the segment does not contain an acknowledgement, so the Acknowledgement number field is ignored. The PSH bit indicates PUSHed data. The receiver is hereby kindly requested to deliver the data to the application upon arrival and not buffer it until a full buffer has been received (which it might otherwise do for efficiency). The RST bit is used to abruptly reset a connection that has become confused due to a host crash or some other reason. It is also used to reject an invalid segment
or refuse an attempt to open a connection. In general, if you get a segment with the RST bit on, you have a problem on your hands. The SYN bit is used to establish connections. The connection request has SYN = 1 and ACK = 0 to indicate that the piggyback acknowledgement field is not in use. The connection reply does bear an acknowledgement, however, so it has SYN = 1 and ACK = 1. In essence, the SYN bit is used to denote both CONNECTION REQUEST and CONNECTION ACCEPTED, with the ACK bit used to distinguish between those two possibilities. The FIN bit is used to release a connection. It specifies that the sender has no more data to transmit. However, after closing a connection, the closing process may continue to receive data indefinitely. Both SYN and FIN segments have sequence numbers and are thus guaranteed to be processed in the correct order.
Flow control in TCP is handled using a variable-sized sliding window. The Window size field tells how many bytes may be sent starting at the byte acknowledged. A Window size field of 0 is legal and says that the bytes up to and including Acknowledgement number − 1 have been received, but that the receiver has not had a chance to consume the data and would like no more data for the moment. The receiver can later grant permission to send by transmitting a segment with the same Acknowledgement number and a nonzero Window size field.
The Options field provides a way to add extra facilities not covered by the regular header. Many options have been defined and several are commonly used. The options are of variable length, fill a multiple of 32 bits by using padding with zeros, and may extend to 40 bytes to accommodate the longest TCP header that can be specified. Some options are carried when a connection is established to negotiate or inform the other side of capabilities. Other options are carried on packets during the lifetime of the connection. Each option has a Type-Length-Value encoding. A widely used option is the one that allows each host to specify the **MSS**
(**Maximum Segment Size**) it is willing to accept. The **window scale** option allows the sender and receiver to negotiate a window scale factor at the start of a connection. Both sides use the scale factor to shift the Window size field up to 14 bits to the left, thus allowing windows of

up to 230 bytes. Most TCP implementations support this option. The **timestamp** option carries a timestamp sent by the sender and echoed by the receiver. It is included in every packet, once its use is established during connection setup, and used to compute round-trip time samples that are used to estimate when a packet has been lost. It is also used as a logical extension of the 32-bit sequence number. On a fast connection, the sequence number may wrap around quickly, leading to possible confusion between old and new data. The **PAWS** (**Protection Against Wrapped Sequence numbers**) scheme discards arriving segments with old timestamps to prevent this problem. Finally, the **SACK** (**Selective ACKnowledgement**) option lets a receiver tell a sender the ranges of sequence numbers that it has received. It supplements the Acknowledgement number and is used after a packet has been lost but subsequent (or duplicate) data has arrived. The new data is not reflected by the Acknowledgement number field in the header because that field gives only the next in-order byte that is expected. With SACK, the sender is explicitly aware of what data the receiver has and hence can determine what data should be retransmitted.

18 b. How transport layer connection is established in TCP? Illustrate with state diagrams.

TCP uses this three-way handshake to establish connections. This establishment protocol involves one peer checking with the other that the connection request is indeed current. The normal setup procedure when host 1 initiates is sown in Fig. 6-11. Host 1 chooses a sequence number , x, and sends a CONNECTION REQUEST segment containing it to host 2. Host 2 replies with an ACK segment acknowledging x and announcing its own initial sequence number, y. Finally, host 1 acknowledges host 2's choice of an initial sequence number in the first data segment that it sends.

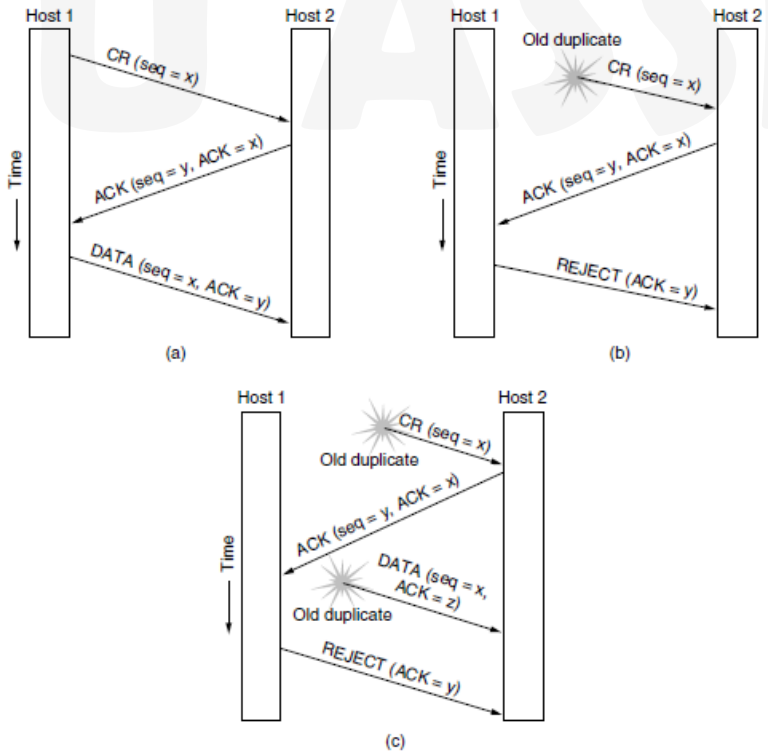


Figure 6-11. Three protocol scenarios for establishing a connection using a three-way handshake. CR denotes CONNECTION REQUEST. (a) Normal operation. (b) Old duplicate CONNECTION REQUEST appearing out of nowhere. (c) Duplicate CONNECTION REQUEST and duplicate ACK.

In Fig. (b), the first segment is a delayed duplicate CONNECTION REQUEST from an old connection. This segment arrives at host 2 without host 1's knowledge. Host 2 reacts to this segment by sending host 1 an ACK segment, in effect asking for verification that host 1 was indeed trying to set up a new connection. When host 1 rejects host 2's attempt to establish a connection, host 2 realizes that it was tricked by a delayed duplicate and abandons the connection. In this way, a delayed duplicate does no damage.
The worst case is when both a delayed CONNECTION REQUEST and an ACK are floating around in the subnet. This case is shown in Fig. As in the previous example, host 2 gets a delayed CONNECTION REQUEST and replies to it. At this point, it is crucial to realize that host 2 has proposed using y as the initial sequence number for host 2 to host 1 traffic, knowing full well that no segments containing sequence number y or acknowledgements to y are still in existence. When the second delayed segment arrives at host 2, the fact that z has been acknowledged rather than y tells host 2 that this, too, is an old duplicate.

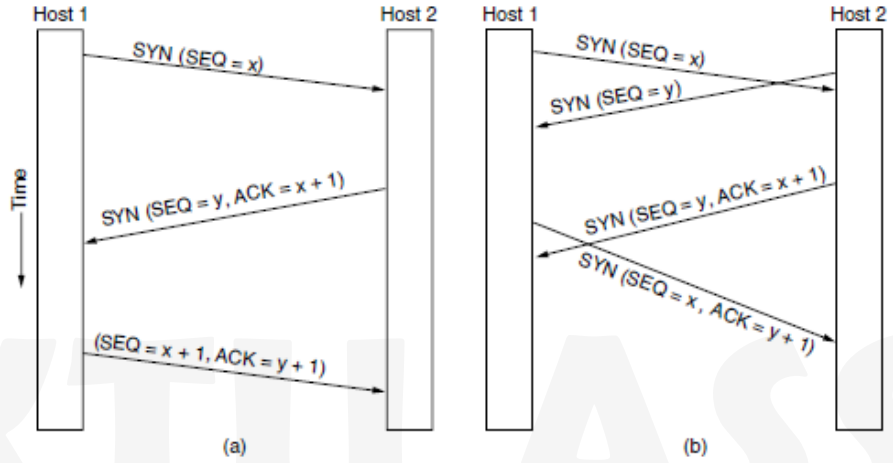


Figure 6-37. (a) TCP connection establishment in the normal case. (b) Simultaneous connection establishment on both sides.

Connections are established in TCP by means of the three-way handshake .To establish a connection, one side, say, the server, passively waits for an incoming connection by executing the LISTEN and ACCEPT primitives in that order, either specifying a specific source or nobody in particular. The other side, say, the client, executes a CONNECT primitive, specifying the IP address and port to which it wants to connect, the maximum TCP segment size it is willing to accept, and optionally some user data (e.g., a password). The CONNECT Primitive sends a TCP segment with the SYN bit on and ACK bit off and waits for a response. When this segment arrives at the destination, the TCP entity there checks to see if there is a process that has done a LISTEN on the port. If not, it sends a reply with the RST bit on to reject the connection. If some process is listening to the port, that process is given the incoming TCP segment. It can either accept or reject the connection. If it accepts, an acknowledgement segment is sent back. The sequence of TCP segments sent in the normal case is shown in Fig. 6-37(a). SYN segment consumes 1 byte of sequence space so that it can be acknowledged unambiguously. In the event that two hosts simultaneously attempt to establish a connection between the same two sockets, the sequence of events is as illustrated in Fig. The result of these events is that just one connection is established, not two, because connections are identified by their end points. If the first setup results in a connection identified by (x, y) and the second one does too, only one table entry is made, namely, for (x, y).

19. Explain the process of address resolution by using ARP and RARP protocol.

Designers of TCP/IP protocols found a creative solution to the address resolution problem for networks like the Ethernet that have broadcast capability. The solution allows new hosts or routers to be added to the network without recompiling code, and does not require maintenance of a centralized database. To avoid maintaining a table of mappings, the designers chose to use a low-level protocol to bind addresses dynamically. Termed the **Address Resolution Protocol (ARP),** the protocol provides a mechanism that is both reasonably efficient and easy to maintain. The Address Resolution Protocol, ARP, allows a host to **find** the physical address of a target host on the same physical network, given only the target's IP address.

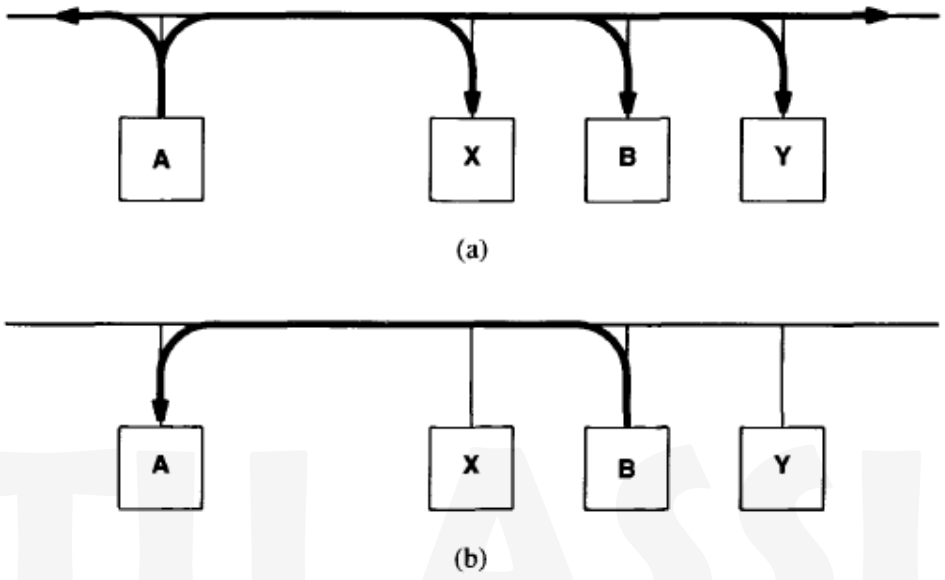


Figure 5.1 The ARP protocol. To determine P_B, B's physical address, from I_B, its IP address, (a) host A broadcasts an ARP request containing I_B to all machines on the net, and (b) host B responds with an ARP reply that contains the pair (I_B, P_B).

As Figure shows, the idea behind dynamic resolution with **ARP** is simple: when host **A** wants to resolve **IP** address **ZB,** it broadcasts a special packet that asks the host with IP address **le** to respond with its physical address, **PB. AU** hosts, including B, receive the request, but only host B recognizes its **IP** address and sends a reply that contains its physical address. When **A** receives the reply, it uses the physical address to send the internet packet directly to B.

To reduce communication costs, computers that use ARP maintain a cache of recently acquired IP-to-physical address bindings. That is, whenever a computer sends an **ARP** request and receives an ARP reply, it saves the **IP** address and corresponding hardware address information in its cache for successive lookups. When transmitting a packet, a computer always looks in its cache for a binding before sending an AFW request. If it finds the desired binding in its ARP cache, the computer need not broadcast on the network. Thus, when two computers on a network communicate, they begin with an ARP request and response, and then repeatedly transfer packets without using **ARP** for each one.

ARP is divided into two parts. The first part maps an IP address to a physical address when sending a packet, and the second part answers requests from other machines. Address resolution for outgoing packets seems straightforward, but small details complicate an implementation. Given a destination IP address the software consults its ARP cache to see if it knows the mapping from IP address to physical address. If it does, the software extracts the physical address, places the data in a frame using that address, and sends the frame. If it does not know the mapping, the software must broadcast an **ARP** request and wait for a reply.
Broadcasting an ARP request to find an address mapping can become complex. The target machine can be down or just too busy to accept the request. If so, the sender may not receive a reply or the reply may be delayed. Because the Ethernet is a besteffort delivery system, the initial ARP broadcast request can also be. Meanwhile, the host must store the original outgoing packet so it can be sent once the address has been resolved. In fact, the host must decide whether to allow other application programs to proceed while it processes an AFW request (most do). If so, the software must handle the case where an application generates additional **ARP** requests for the same address without broadcasting multiple requests for a given target. When **ARP** messages travel from one machine to another, they must be carried in physical frames. To identify the frame as carrying an ARP message, the sender assigns a special value to the type field in the frame header, and places the ARP message in the frame's data field. When a frame arrives at a computer, the network software uses the frame type to determine its contents.

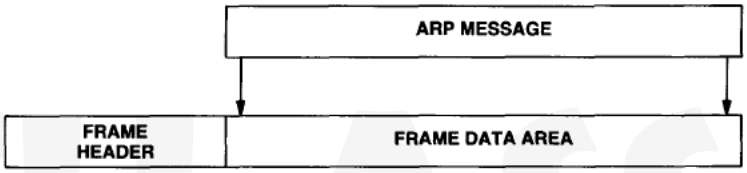


Figure 5.2 An ARP message encapsulated in a physical network frame.

ARP Protocol Format

ARP packets does not have a fixed-format header. Instead, to make ARP useful for a variety of network technologies, the length of fields that contain addresses depend on the type of network. Field **HARDWARE TYPE** specifies a hardware interface type for which the sender seeks an answer; it contains the value **1** for Ethernet. Similarly, field **PROTOCOL** TYPE specifies the type of high-level protocol address the sender has supplied; it contains **0800,,** for **IP** addresses. Field **OPERATION** specifies an **ARP** request (I), **ARP** response **(2), RARP** request **(3),** or RARP response (4). Fields **HLEN** and **PLEN** allow **ARP** to be used with arbitrary networks because they specify the length of the hardware address and the length of the high-level protocol address. The sender supplies its hardware address and **IF'** address, if known, in fields **SENDER HA** and **SENDER** IP. When making a request, the sender also supplies the target hardware address **(RARP)** or target IP address (ARP), using fields **TARGET** HA or **TARGET IP.** Before the target machine responds, it fills in the missing addresses, swaps the target and sender pairs, and changes the operation to a reply. Thus, a reply carries the **IP** and hardware addresses of the original requester, as well as the **IP** and hardware addresses of the machine for which a binding was sought.

0	8	16	24	31
HARDWARE TYPE		PROTOCOL TYPE		
HLEN	PLEN	OPERATION		
SENDER HA (octets 0-3)				
SENDER HA (octets 4-5)		SENDER IP (octets 0-1)		
SENDER IP (octets 2-3)		TARGET HA (octets 0-1)		
TARGET HA (octets 2-5)				
TARGET IP (octets 0-3)				

Figure 5.3 An example of the ARP/RARP message format when used for IP-to-Ethernet address resolution. The length of fields depends on the hardware and protocol address lengths, which are 6 octets for an Ethernet address and 4 octets for an IP address.

TCP/IP protocol that allows a computer to obtain its IP address from a server is known as the **Reverse Address Resolution Protocol (RARP).** Like an **ARP** message, a RARP message is sent from one machine to another encapsulated in the data portion of a network frame. For example, an Ethernet frame carrying a RARP request has the usual preamble, Ethernet source and destination addresses, and packet **type** fields in front of the frame. The frame **type** contains the value 8035,, to identify the contents of the frame **as** a RARP message. The data portion of the frame contains the 28-octet RARP message.

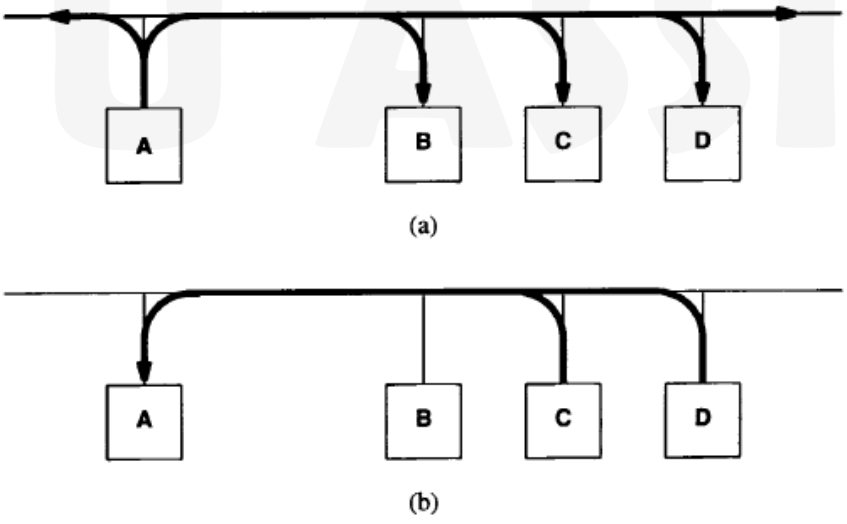


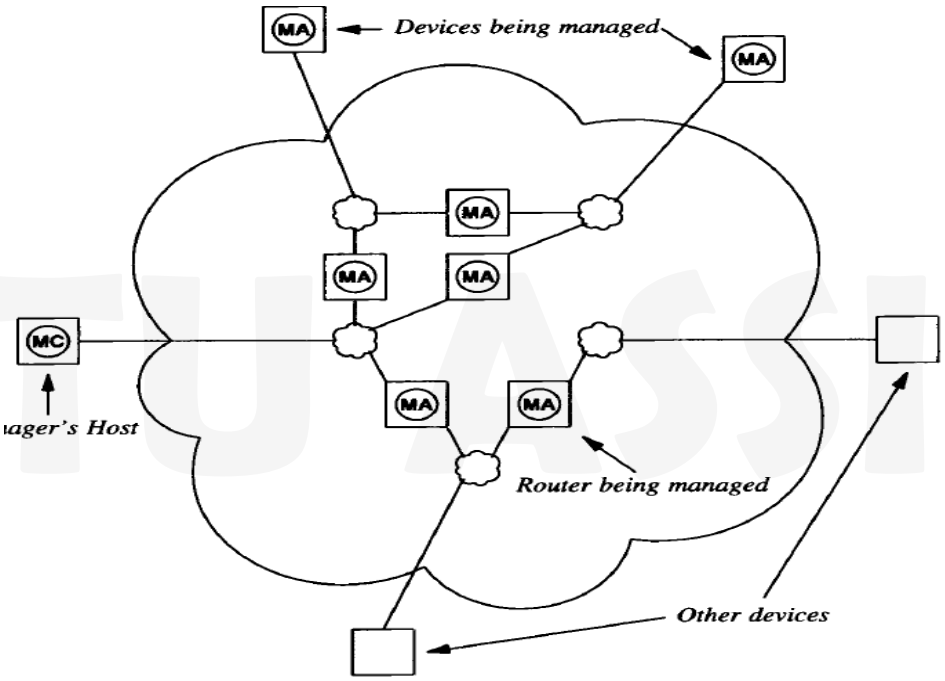
Figure 6.1 Example exchange using the RARP protocol. (a) Machine *A* broadcasts a RARP request specifying itself as a target, and (b) those machines authorized to supply the RARP service (*C* and *D*) reply directly to *A*.

Figure illustrates how a host uses RARP. The sender broadcasts a RARP request that specifies itself **as** both the sender and target machine, and supplies its physical network address in the target hardware address field. All computers on the network receive the request, but only those authorized to supply the RARP service process the request and send a reply; such computers are known informally **as** RARP **servers.** For **RARP** to succeed, the network must contain at least one RARP server. Servers answer requests by filling in the target protocol address field,

changing the message **type** from **request** to **reply,** and sending the reply back directly to the machine making the request. The original machine receives replies from all RARP servers, even though only the first is needed.

20. Explain how network is managed by using SNMP protocol.

Although the designers provided a range of values to be used as the delay between successive router advertisements, they chose the default of 10 minutes. The value was selected as a compromise between rapid failure detection and low overhead. A smaller value would allow more rapid detection of router failure, but would increase network traffic; a larger value would decrease traffic, but would delay failure detection. One of the issues the designers considered was how to accommodate a large number of routers on the same network.From the point of view of a host, the default delay has a severe disadvantage: a host cannot afford to wait many minutes for an advertisement when it first boots. To avoid such delays, the designers included an **ICMP router solicitation message that allows** host to request an immediate advertisement



- Client software usually runs on the manager's workstation.
- Each participating router or host runs a server program.
- Server software is called a management agent or merely an agent.
- A manager invokes client software on the local host computer and specifies an agent with which it communicates.
- After the client contacts the agent, it sends queries to obtain information or it sends commands to change conditions in the router.
- Internet management software uses an authentication mechanism to ensure only authorized managers can access or control a particular device.

- TCP/IP network management protocols divide the management problem into two parts and specify separate standards for each part.
- The first part concerns communication of information.
    - A protocol specifies how client software running on a manager's host **communicates with an agent**.
    - The protocol defines the **format and meaning** of messages clients an d servers exchange as well as the form of names and addresses.
- The second part concerns the data being managed.
    - A protocol specifies which **data items** a managed device must keep as well as the **name of each data item and the syntax** used to express the name.
- Management Information Base (MIB), the standard specifies the data items a managed device must keep, the operations allowed on each, and the meanings.
    - **Router** keeps statistics on the status of its network interfaces,incoming and outgoing packet traffic, dropped datagrams, and error messages generated.
    - **Modem** keeps statistics about the number of characters sent and received, baud rate,and calls accepted.

MIB category	Includes Information About
system	The host or router operating system
interfaces	Individual network interfaces
at	Address translation (e.g., ARP mappings)
ip	Internet Protocol software
icmp	Internet Control Message Protocol software
tcp	Transmission Control Protocol software
udp	User Datagram Protocol software
ospf	Open Shortest Path First software
bgp	Border Gateway Protocol software
rmon	Remote network monitoring
rip-2	Routing Information Protocol software
dns	Domain Name System software

Figure 30.2 Example categories of MIB information. The category is encoded in the identifier used to specify an object.

MIB Variable	Category	Meaning
sysUpTime	system	Time since last reboot
ifNumber	interfaces	Number of network interfaces
ifMtu	interfaces	MTU for a particular interface
ipDefaultTTL	ip	Value IP uses in time-to-live field
ipInReceives	ip	Number of datagrams received
ipForwDatagrams	ip	Number of datagrams forwarded
ipOutNoRoutes	ip	Number of routing failures
ipReasmOKs	ip	Number of datagrams reassembled
ipFragOKs	ip	Number of datagrams fragmented
ipRoutingTable	ip	IP Routing table
icmpInEchos	icmp	Number of ICMP Echo Requests received
tcpRtoMin	tcp	Minimum retransmission time TCP allows
tcpMaxConn	tcp	Maximum TCP connections allowed
tcpInSegs	tcp	Number of segments TCP has received
udpInDatagrams	udp	Number of UDP datagrams received

Figure 30.3 Examples of MIB variables along with their categories.

- SMI standard specifies a set of rules used to define and identify MIB variables.
- To keep network management protocols simple, the SMI places restrictions on the **types of variables allowed** in the MIB, specifies the **rules for naming** those variables, and creates **rules for defining variable** types.
- The SMI standard specifies that all MIB variables must be defined and referenced using ISO's ASN.1.
- ASN.1 is a formal language that has two main features:
    - a notation used in documents that humans read and
    - a compact encoded representation of the same information used in communication protocols.
- In both cases, the precise, formal notation **removes any possible ambiguities** from both the representation and meaning.
- ASN.l also helps **simplify** the implementation of network management protocols and **guarantees interoperability**.
- It defines precisely how to encode both names and data items in a message.
- Names used for MIB variables are taken from the object identifier namespace administered by IS0 and ITU.
- The object identifier namespace is absolute (global), meaning that names are structured to make them globally unique.
- The object identifier namespace is hierarchical.
- The root of the object identifier hierarchy is unnamed, but has three direct descendants managed by: ISO, ITU, and jointly by IS0 and ITU.
- The descendants are assigned both short text strings and integers that identify them.

- IS0 has allocated one subtree for use by other national or international standards organizations (including U.S. standards organizations)
- The U.S. National Institute for Standards and Technology has allocated a subtree for the U.S. **Department of Defense**.
- the names of all MIB variables corresponding to IP have an identifier that begins with the prefix 1.3.6.1.2.1.4
- Textual label representation instead of the numeric representation can be:

  iso . org . dod. internet. mgmt . mib . Ip
- Suffix 0 refers to the instance of the variable with that name. So, when it appears in a message sent to a router, the numeric representation is: 1.3.6.1.2.1.4.3.0

  which refers to the instance of on that router.
- Network management protocols specify communication between the network management client program a manager invokes and a network management server program executing on a host or router.
- In addition to defining the form and meaning of messages exchanged and the representation of names and values in those messages, network management protocols also define administrative relationships among routers being managed.
- They provide authentication of managers.

Command	Meaning
get-request	Fetch a value from a specific variable
get-next-request	Fetch a value without knowing its exact name
get-bulk-request	Fetch a large volume of data (e.g., a table)
response	A response to any of the above requests
set-request	Store a value in a specific variable
inform-request	Reference to third-part data (e.g., for a proxy)
snmpv2-trap	Reply triggered by an event
report	Undefined at present

Figure 30.6 The set of possible SNMP operations. *Get-next-request* allows the manager to iterate through a table of items.

- Operations get-request and set-request provide the basic fetch and store operations response provides the reply.
- SNMP specifies that operations must be atomic, meaning that if a single SNMP message specifies operations on multiple variables, the server either performs all operations or none of them.
- The get-next-request operation allows a client to iterate through a table without knowing how many items the table contains.
- When sending a get-next-request, the client supplies a prefix of a valid object identifier, P.

- The agent examines the set of object identifiers for all variables it controls, and sends a response for the variable that occurs next in lexicographic order.
- The agent must know the ASN.1 names of all variables and be able to select the first variable with object identifier greater than P.
- SNMP messages do not have fixed fields.
- They use the standard ASN.1 encoding.
- Message can be difficult for humans to decode and understand.
- Each item in the grammar consists of a descriptive name followed by a declaration of the item's type.
- For example, an item such as

  msgversion INTEGER (0..2147483647)

  declares the name msgversion to be a nonnegative integer less than or equal to 2147483647.

```
SNMPv3Message ::=
    SEQUENCE {
        msgVersion  INTEGER (0..2147483647),
          -- note: version number 3 is used for SNMPv3
        msgGlobalData  HeaderData,
        msgSecurityParameters  OCTET STRING,
        msgData  ScopedPduData
    }
```

Figure 30.7 The SNMP message format in ASN.1-style notation. Text following two consecutive dashes is a comment.

21. Explain the architecture of World Wide Web.

The Web, as the World Wide Web is popularly known, is an architectural framework for accessing linked content spread out over millions of machines all over the Internet. From the users' point of view, the Web consists of a vast, worldwide collection of content in the form of **Web pages**, often just called **pages** for short. Each page may contain links to other pages anywhere in the world. Users can follow a link by clicking on it, which then takes them to the page pointed to. This process can be repeated indefinitely. The idea of having one page point to another, now called **hypertext.** Pages are generally viewed with a program called a **browser**. A piece of text, icon, image, and so on associated with another page is called a **hyperlink**. To follow a link, the user places the mouse cursor on the linked portion of the page area (which causes the cursor to change shape) and clicks. Following a link is simply a way of telling the browser to fetch another page. In the early days of the Web, links were highlighted with underlining and colored text so that they would stand out. Nowadays, the creators of Web pages have ways to control the look of linked regions, so a link might appear as an icon or change its appearance .when the mouse passes over it. It is up to the creators of the page to make the links

visually distinct, to provide a usable interface. The basic model behind the display of pages is also shown in Fig. The browser is displaying a Web page on the client machine. Each page is fetched by sending a request to one or more servers, which respond with the contents of the page. The request-response protocol for fetching pages is a simple text-based protocol that runs over TCP, just as was the case for SMTP. It is called **HTTP** (**HyperText Transfer Protocol**). The content may simply be a document that is
read off a disk, or the result of a database query and program execution. The page is a **static page** if it is a document that is the same every time it is displayed. In contrast, if it was generated on demand by a program or contains a program it is a **dynamic page**. A dynamic page may present itself differently each time it is displayed.

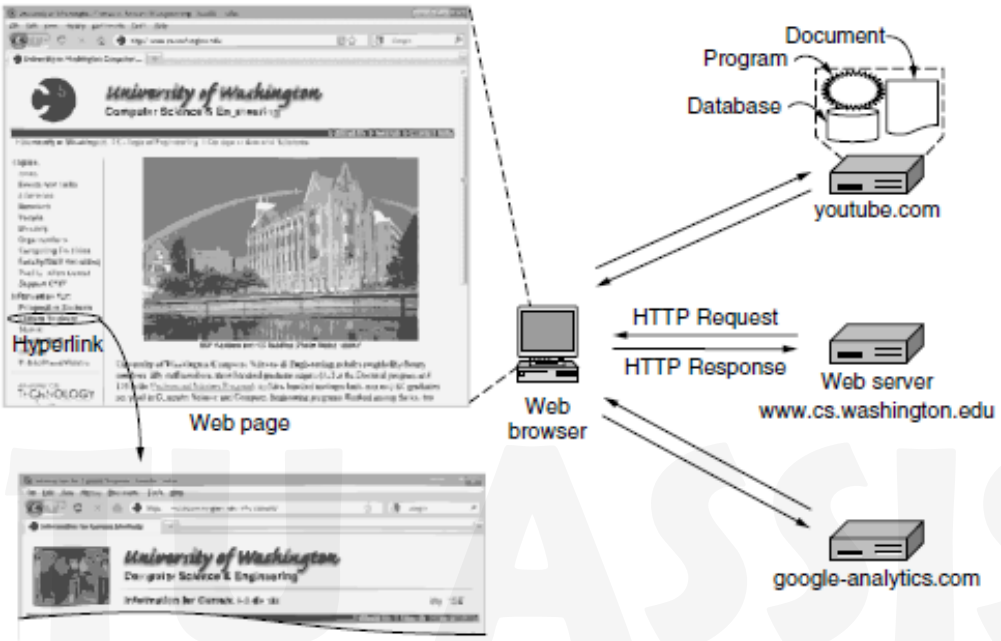


Figure 7-18. Architecture of the Web.

The Client Side

When the Web was first created, it was immediately apparent that having one page point to another Web page required mechanisms for naming and locating pages. In particular, three questions had to be answered before a selected page could be displayed:
1. What is the page called?
2. Where is the page located?
3. How can the page be accessed?

If every page were somehow assigned a unique name, there would not be any ambiguity in identifying pages. The solution chosen identifies pages in a way that solves all three problems at once. Each page is assigned a **URL** (**Uniform Resource Locator**) that effectively serves as the page's worldwide name. URLs have three parts: the protocol (also known as the **scheme**), the DNS name of the machine on which the page is located, and the path uniquely indicating the specific page (a file to read or program to run on the machine). In the general case, the path has a hierarchical name that models a file directory structure. This URL consists of three parts: the protocol (http), the DNS name of the host (www.cs.washington.edu), and the path name (index.html).

When a user clicks on a hyperlink, the browser carries out a series of steps in order to fetch the page pointed to. Let us trace the steps that occur when our example link is selected:
1. The browser determines the URL (by seeing what was selected).
2. The browser asks DNS for the IP address of the server www.cs.washington.edu.
3. DNS replies with 128.208.3.88.
4. The browser makes a TCP connection to 128.208.3.88 on port 80, the well-known port for the HTTP protocol.
5. It sends over an HTTP request asking for the page /index.html.

6. The www.cs.washington.edu server sends the page as an HTTP response, for example, by sending the file /index.html.
7. If the page includes URLs that are needed for display, the browser fetches the other URLs using the same process. In this case, the URLs include multiple embedded images also fetched from www.cs.washington.edu, an embedded video from youtube.com, and a script from google-analytics.com.
8. The browser displays the page /index.html as it appears in Fig.
9. The TCP connections are released if there are no other requests to the same servers for a short period. The URL design is open-ended in the sense that it is straightforward to have browsers use multiple protocols to get at different kinds of resources. The http protocol is the Web's native language, the one spoken by Web servers. **HTTP** stands for **HyperText Transfer Protocol**. We will examine it in more detail later in this section. The ftp protocol is used to access files by FTP, the Internet's file transfer protocol. It is possible to access a local file as a Web page by using the file protocol, or more simply, by just naming it. This approach does not require having a server. Of course, it works only for local files, not remote ones. The mailto protocol does not really have the flavor of fetching Web pages, but is useful anyway. It allows users to send email from a Web browser. Most browsers will respond when a mailto link is followed by starting the user's mail agent to compose a message with the address field already filled in. The rtsp and sip protocols are for establishing streaming media sessions and audio and video calls. Finally, the about protocol is a convention that provides information about the browser. URLs have been generalized into **URIs** (**Uniform Resource Identifiers**). Some URIs tell how to locate a resource. These are the URLs. Other URIs tell the name of a resource but not where to find it. These URIs are called **URN**s (**Uniform Resource Names**). The rules for writing URIs are given in RFC 3986, while the different URI schemes in use are tracked by IANA.

MIME Types

To allow all browsers to understand all Web pages, Web pages are written in a standardized language called HTML. Although a browser is basically an HTML interpreter, most browsers have numerous buttons and features to make it easier to navigate the Web. For a rapidly growing collection of file types, most browsers have chosen a more general solution. When a server returns a page, it also returns some additional information about the page. This information includes the MIME type of the page . Pages of type text/html are just displayed directly, as are pages in a few other built-in types. If the MIME type is not one of the built-in ones, the browser consults its table of MIME types to determine how to display the page.
This table associates MIME types with viewers. There are two possibilities: plug-ins and helper applications. A **plug-in** is a third-party code module that is installed as an extension to the browser, as illustrated in Fig. Because plug-ins run inside the browser, they have access to the current page and can modify its appearance. Each browser has a set of procedures that all plug-ins must implement so the browser can call the plug-ins. For example, there is typically a procedure the browser's base code calls to supply the plug-in with data to display. This set of

procedures is the plug-in's interface and is browser specific.In addition, the browser makes a set of its own procedures available to the plug-in, to provide services to plug-ins. Typical procedures in the browser interface are for allocating and freeing memory, displaying a message on the browser's status line, and querying the browser about parameters.

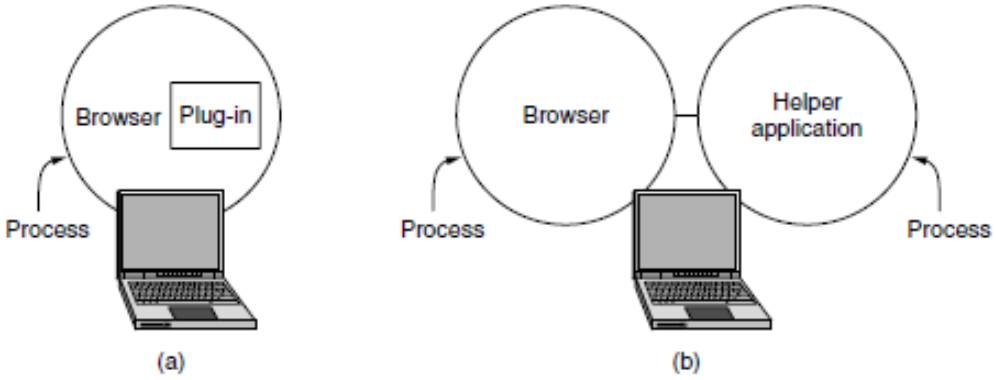


Figure 7-20. (a) A browser plug-in. (b) A helper application.

Before a plug-in can be used, it must be installed. The usual installation procedure is for the user to go to the plug-in's Web site and download an installation file. Executing the installation file unpacks the plug-in and makes the appropriate calls to register the plug-in's MIME type with the browser and associate the plug-in with it. Browsers usually come preloaded with popular plug-ins. The other way to extend a browser is make use of a **helper application**. This is a complete program, running as a separate process.

The Server Side

the steps that the server performs in its main loop are:
1. Accept a TCP connection from a client (a browser).
2. Get the path to the page, which is the name of the file requested.
3. Get the file (from disk).
4. Send the contents of the file to the client.
5. Release the TCP connection.

Web servers are implemented with a different design to serve many requests per second. One problem with the simple design is that accessing files is often the bottleneck. Disk reads are very slow compared to program execution, and the same files may be read repeatedly from disk using operating system calls. Another problem is that only one request is processed at a time. The file may be large, and other requests will be blocked while it is transferred. One obvious improvement is to maintain a cache in memory of the n most recently read files or a certain number of gigabytes of content. Before going to disk to get a file, the server checks the cache. If the file is there, it can be served directly from memory, thus eliminating the disk access. Although effective caching requires a large amount of main memory and some extra processing time to check the cache and manage its contents, the savings in time are nearly always worth the overhead and expense. To tackle the problem of serving a single request at a time, one strategy is to make the server **multithreaded**. In one design, the server consists of a front-end module that accepts all incoming requests and k processing modules, as shown in Fig. The threads all belong to the same process, so the processing modules all have access to the cache

within the process' address space. When a request comes in, the front end accepts it and builds a short record describing it. It then hands the record to one of the processing modules.

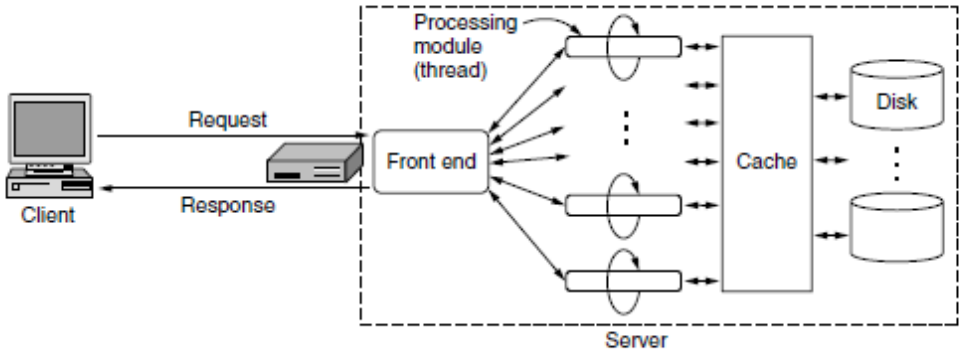


Figure 7-21. A multithreaded Web server with a front end and processing modules.

The processing module first checks the cache to see if the file needed is there. If so, it updates the record to include a pointer to the file in the record. If it is not there, the processing module starts a disk operation to read it into the cache (possibly discarding some other cached file(s) to make room for it). When the file comes in from the disk, it is put in the cache and also sent back to the client. The advantage of this scheme is that while one or more processing modules are blocked waiting for a disk or network operation to complete (and thus consuming no CPU time), other modules can be actively working on other requests. With k processing modules, the throughput can be as much as k times higher than with a single-threaded server. In fact, the actual processing of each request can get quite complicated. For this reason, in many servers each processing module performs a series of steps. The front end passes each incoming request to the first available module, which then carries it out using some subset of the following steps, depending on which ones are needed for that particular request. These steps occur after the TCP connection and any secure transport mechanism have been established.

1. Resolve the name of the Web page requested.
2. Perform access control on the Web page.
3. Check the cache.
4. Fetch the requested page from disk or run a program to build it.
5. Determine the rest of the response (e.g., the MIME type to send).
6. Return the response to the client.
7. Make an entry in the server log.

Cookies

Navigating the Web as we have described it so far involves a series of independent page fetches. There is no concept of a login session. The browser sends a request to a server and gets back a file. Then the server forgets that it has ever seen that particular client. This model is perfectly adequate for retrieving publicly available documents, and it worked well when the Web was first created. However, it is not suited for returning different pages to different users depending on what they have already done with the server. This behavior is needed for many ongoing interactions with Web sites . The name cookies derives from ancient programmer slang in which a program calls a procedure and gets something back that it may need to present later to get some work done in 1994 and are now specified in RFC 2109.

When a client requests a Web page, the server can supply additional information in the form of a cookie along with the requested page. The cookie is a rather small, named string (of at

most 4 KB) that the server can associate with a browser. This association is not the same thing as a user, but it is much closer and more useful than an IP address. Browsers store the offered cookies for an interval, usually in a cookie directory on the client's disk so that the cookies persist across browser invocations, unless the user has disabled cookies. Cookies are just strings, not executable programs. In principle, a cookie could contain a virus, but since cookies are treated as data, there is no official way for the virus to actually run and do damage. However, it is always possible for some hacker to exploit a browser bug to cause activation. A cookie may contain up to five fields, as shown in Fig. The Domain tells where the cookie came from. Browsers are supposed to check that servers are not lying about their domain. Each domain should store no more than 20 cookies per client. The Path is a path in the server's directory structure that identifies which parts of the server's file tree may use the cookie. The Content field takes the form name = value. Both name and value can be anything the server wants. This field is where the cookie's content is stored. The Expires field specifies when the cookie expires. If this field is absent, the browser discards the cookie when it exits. Such a cookie is called a **nonpersistent cookie**. If a time and date are supplied, the cookie is said to be a **persistent cookie** and is kept until it expires. Expiration times are given in Greenwich Mean Time. To remove a cookie from a client's hard disk, a server just sends it again, but with an expiration time in the past. Finally, the Secure field can be set to indicate that the browser may only return the cookie to a server using a secure transport, namel